Computer Graphics

Lecture 06

Viewing and clipping

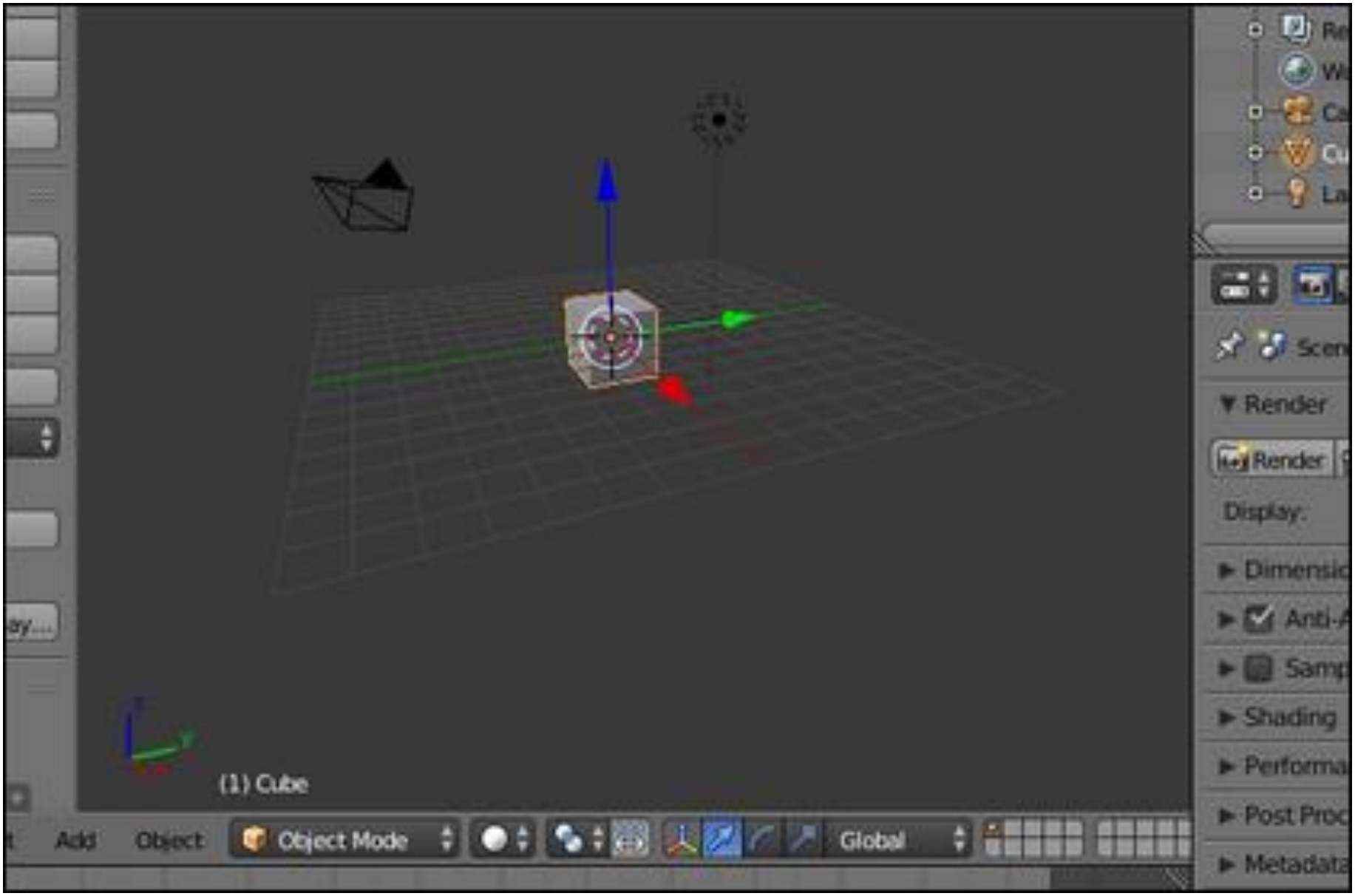
Outlines

1. **Viewing**
   - Camera

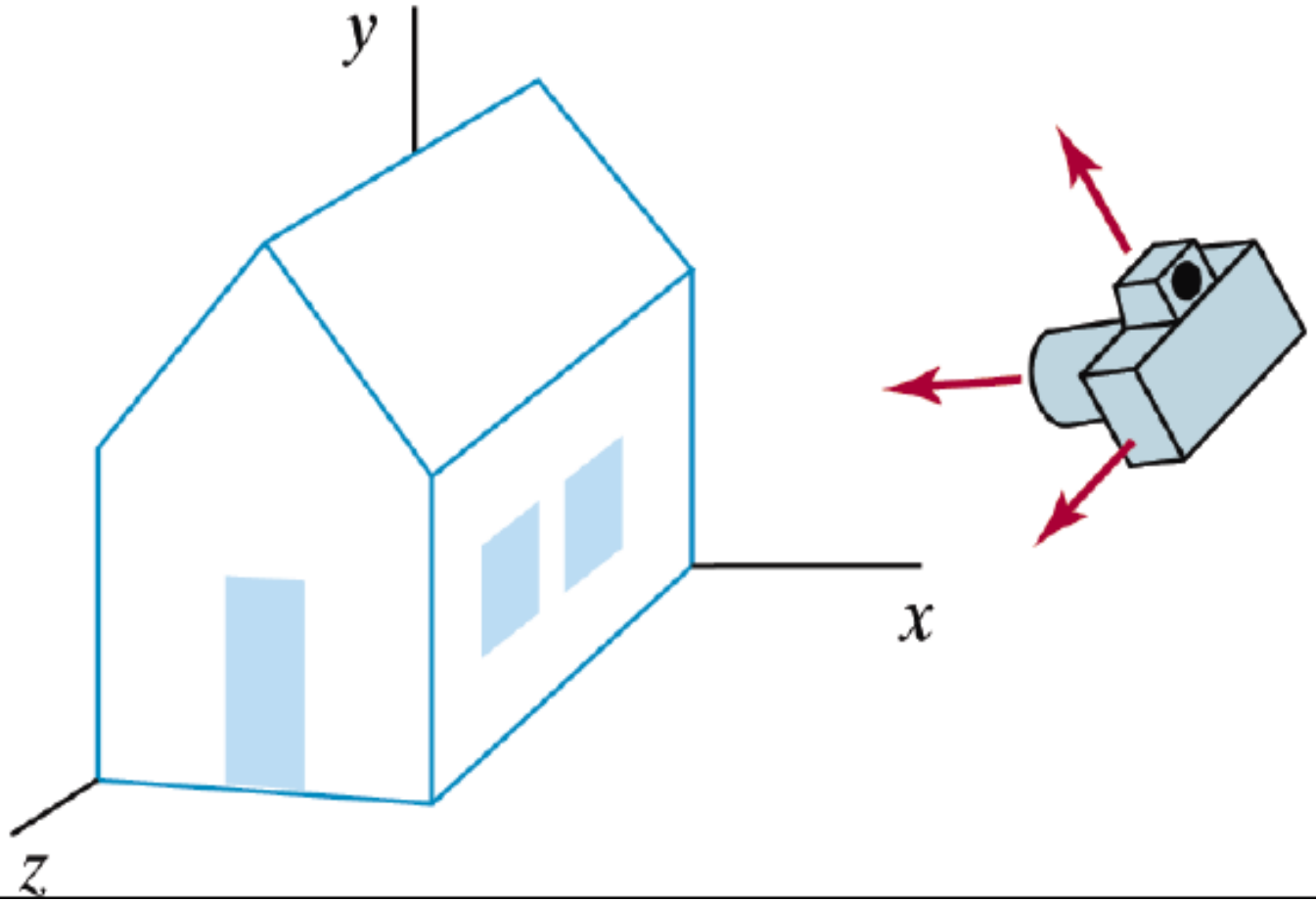
2. Clipping

- It is removing lines or portions of lines outside an area
  - Line
    - Cohen– Algorithm
  - Polygon
    - Sutherland Hodgman - Algorithm

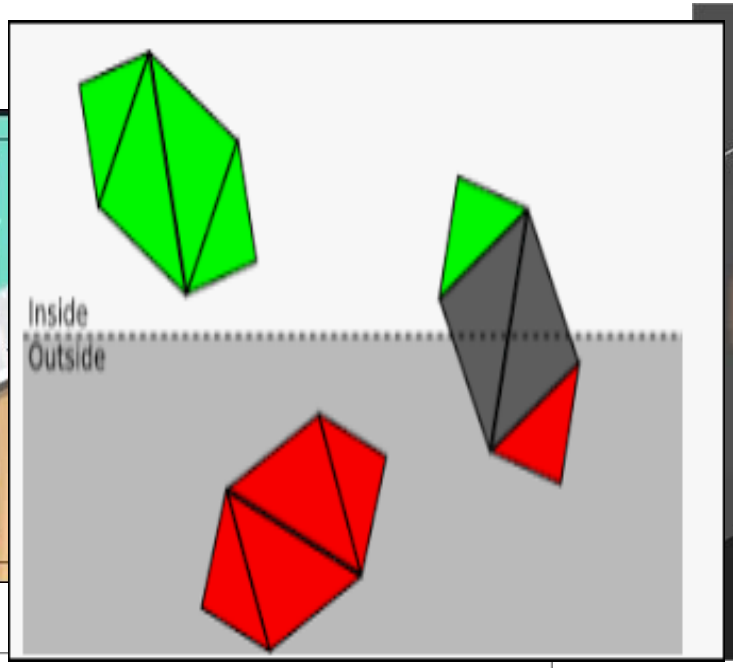
- Light Camera

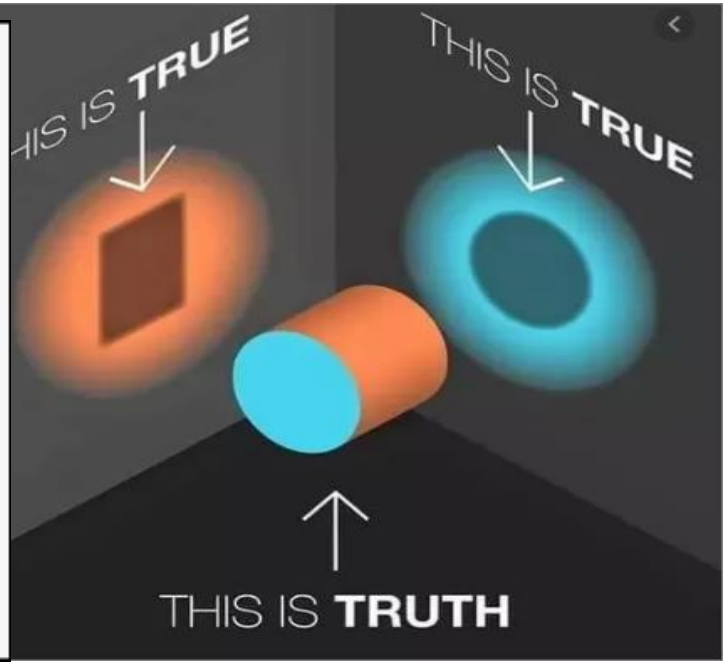


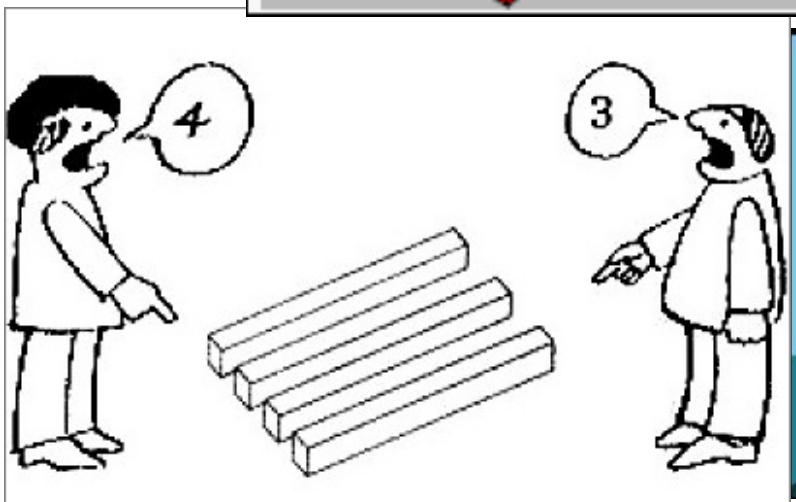
Remember The Big Idea

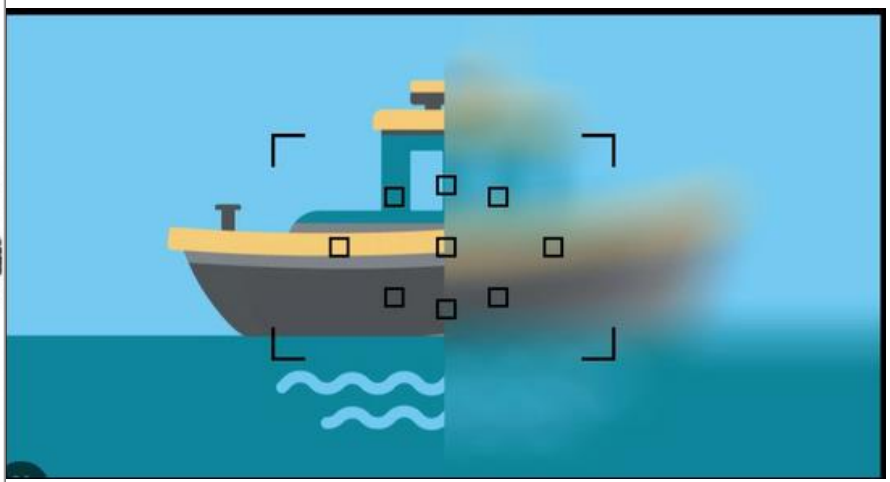


- Camera
  - Viewing - Clipping – Focus - Zoom





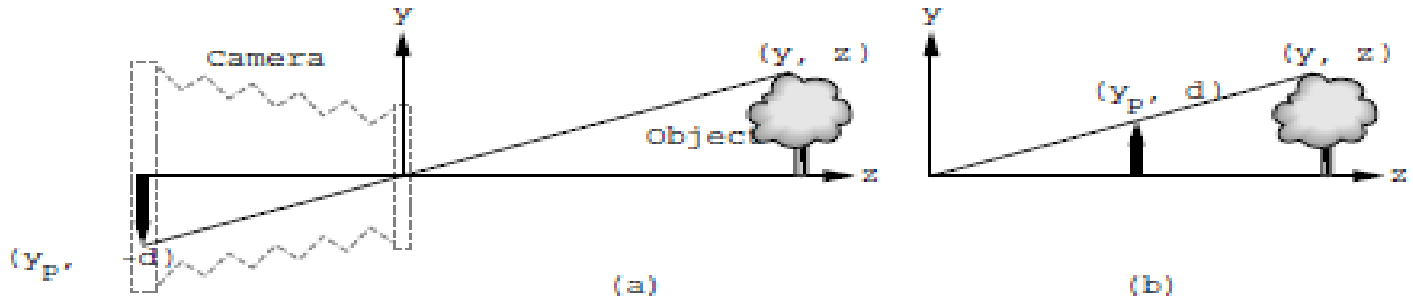




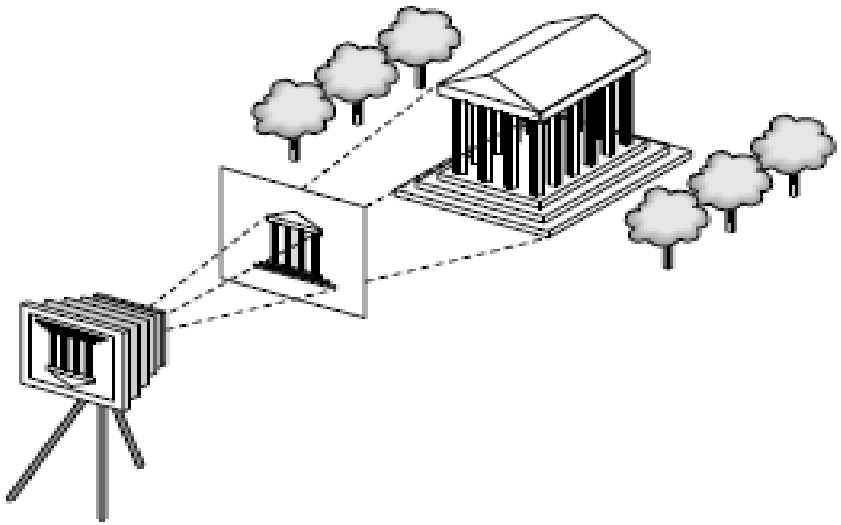
1- Camera model

The synthetic camera model

In computer graphics we use a **synthetic camera model** to mimic the behaviour of a real camera. The image in a pinhole camera is inverted. The film plane is behind the lens.

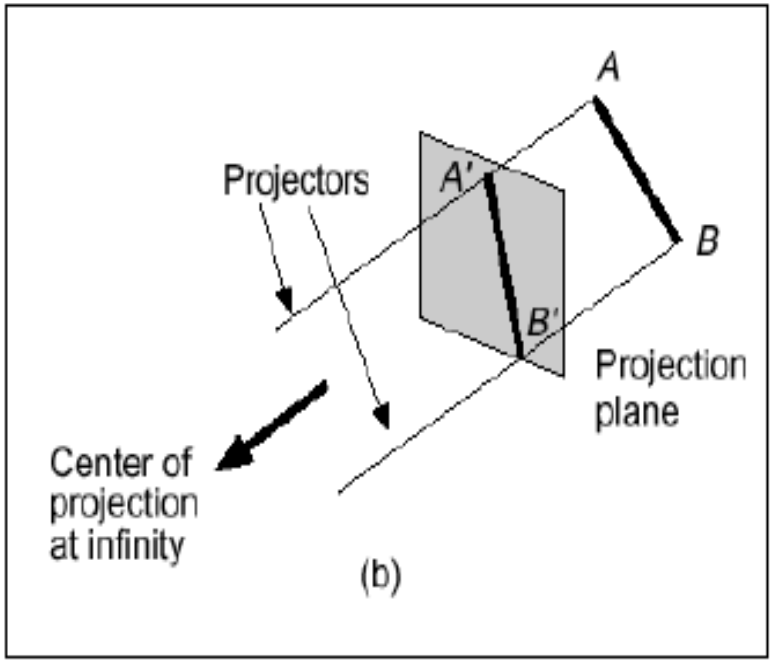


In the synthetic camera model we avoid the inversion by placing the film plane, called the **projection plane**, in front of the lens.

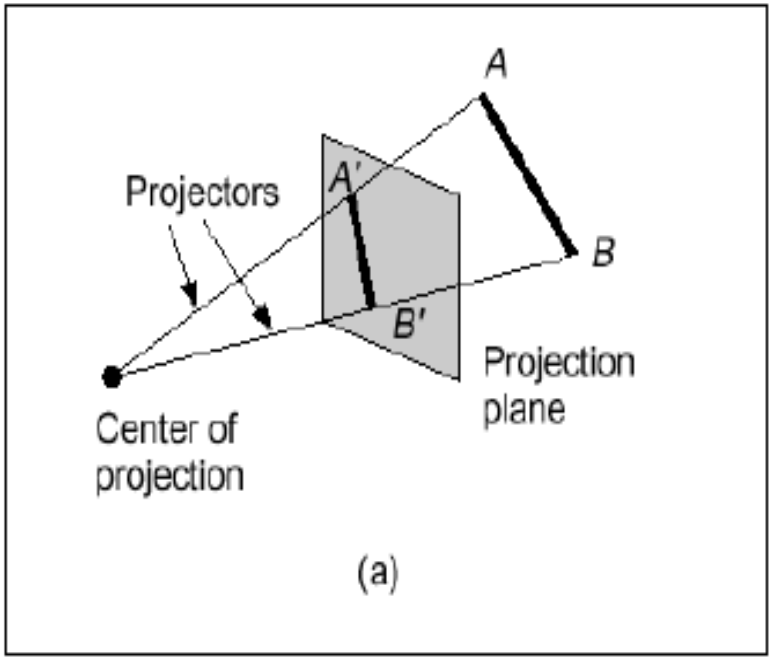


There are two broad classes of projection:

- Parallel: Typically used for architectural and engineering drawings
- Perspective: Realistic looking and used in computer graphics

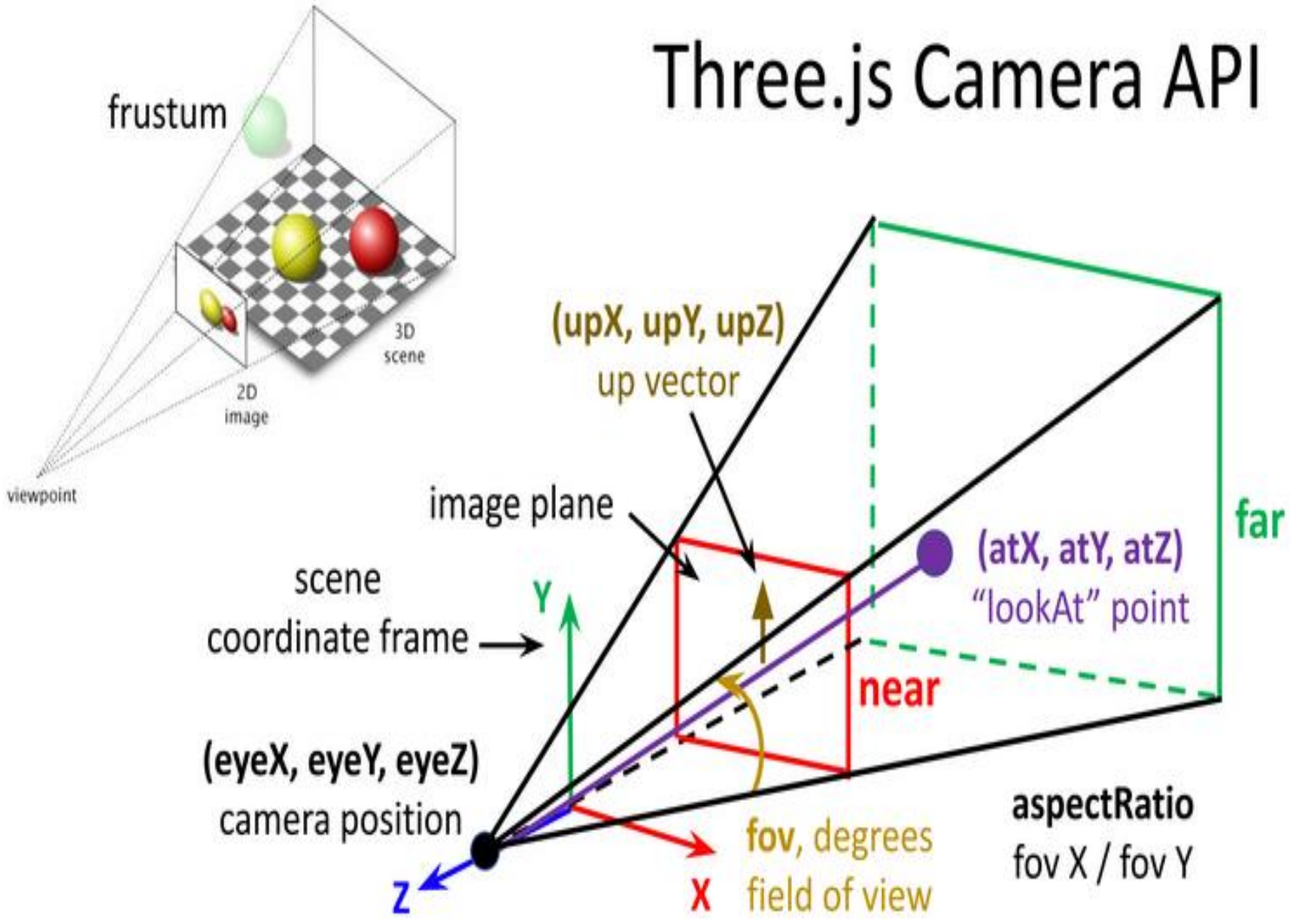


Parallel Projection



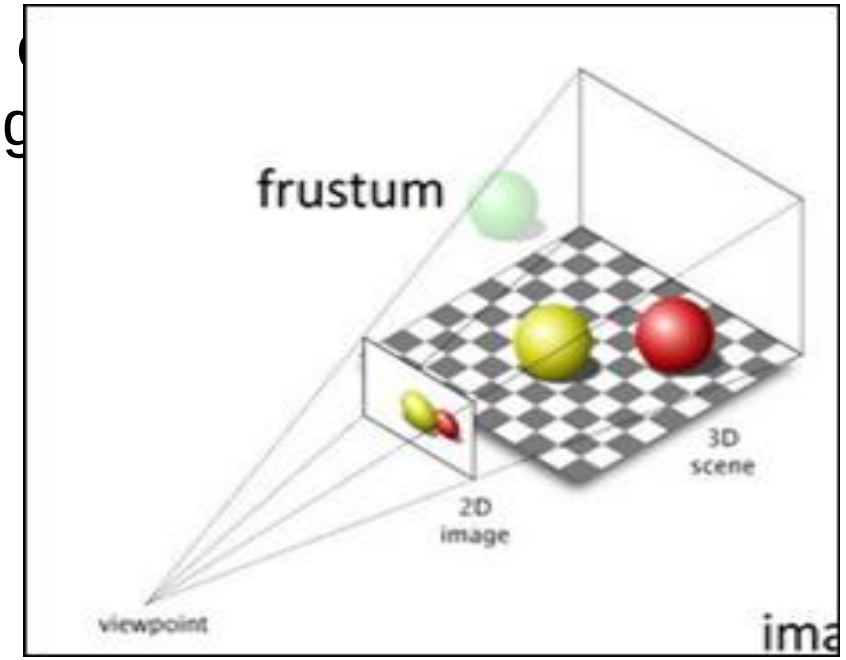
Perspective Projection

The diagram below summarizes the key elements of the camera setup:



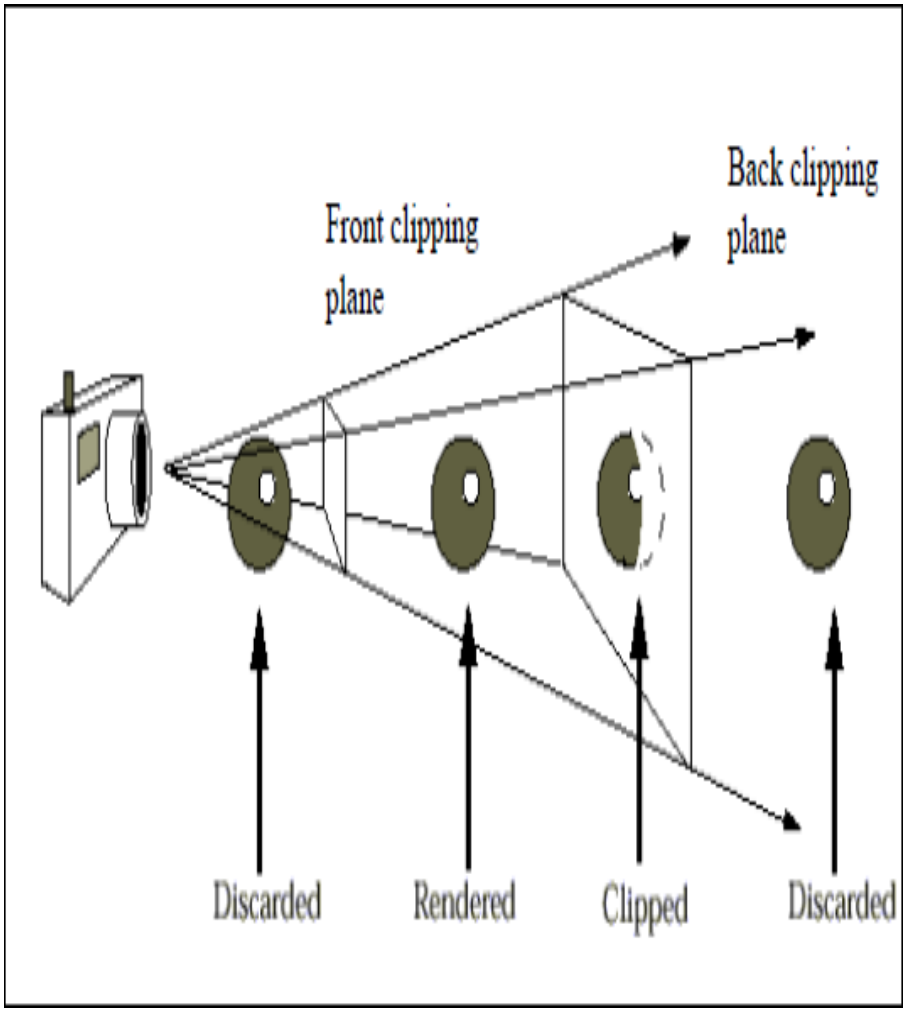
view frustum

- It is the region of space in the modeled world that may appear on the screen; it is the field of view of a perspective virtual camera system.
- ***Front* and *back*** *clipping planes:* limit extent of camera's view by rendering (parts of) and throwing away everything



1- Length →front and back clipping planes

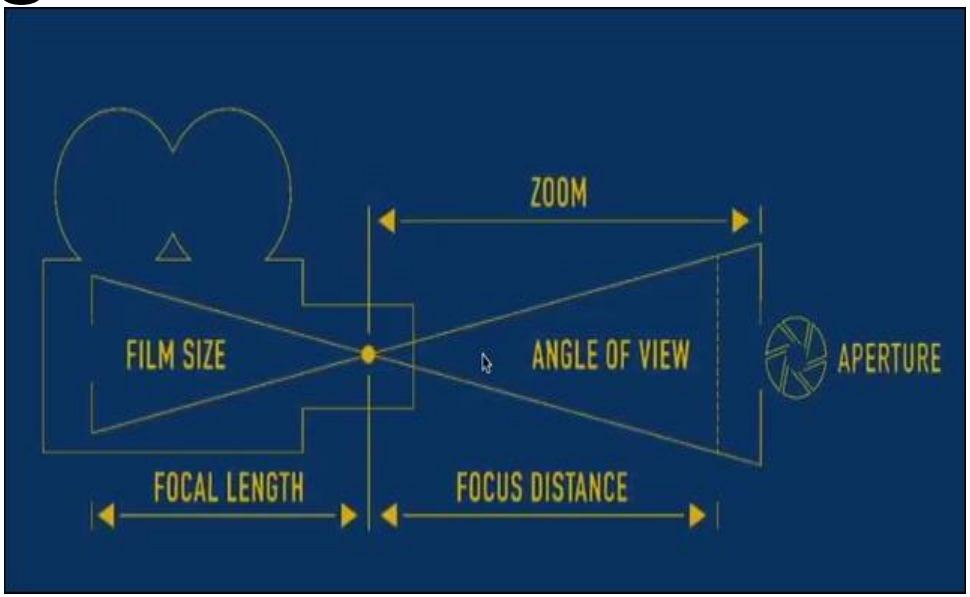
- Volume of space between *Front* and *Back clipping planes* defines what camera can see
  - Position of planes defined by distance along *Look vector*
  - Objects appearing outside of view volume don't get drawn
  - Objects intersecting view volume get clipped

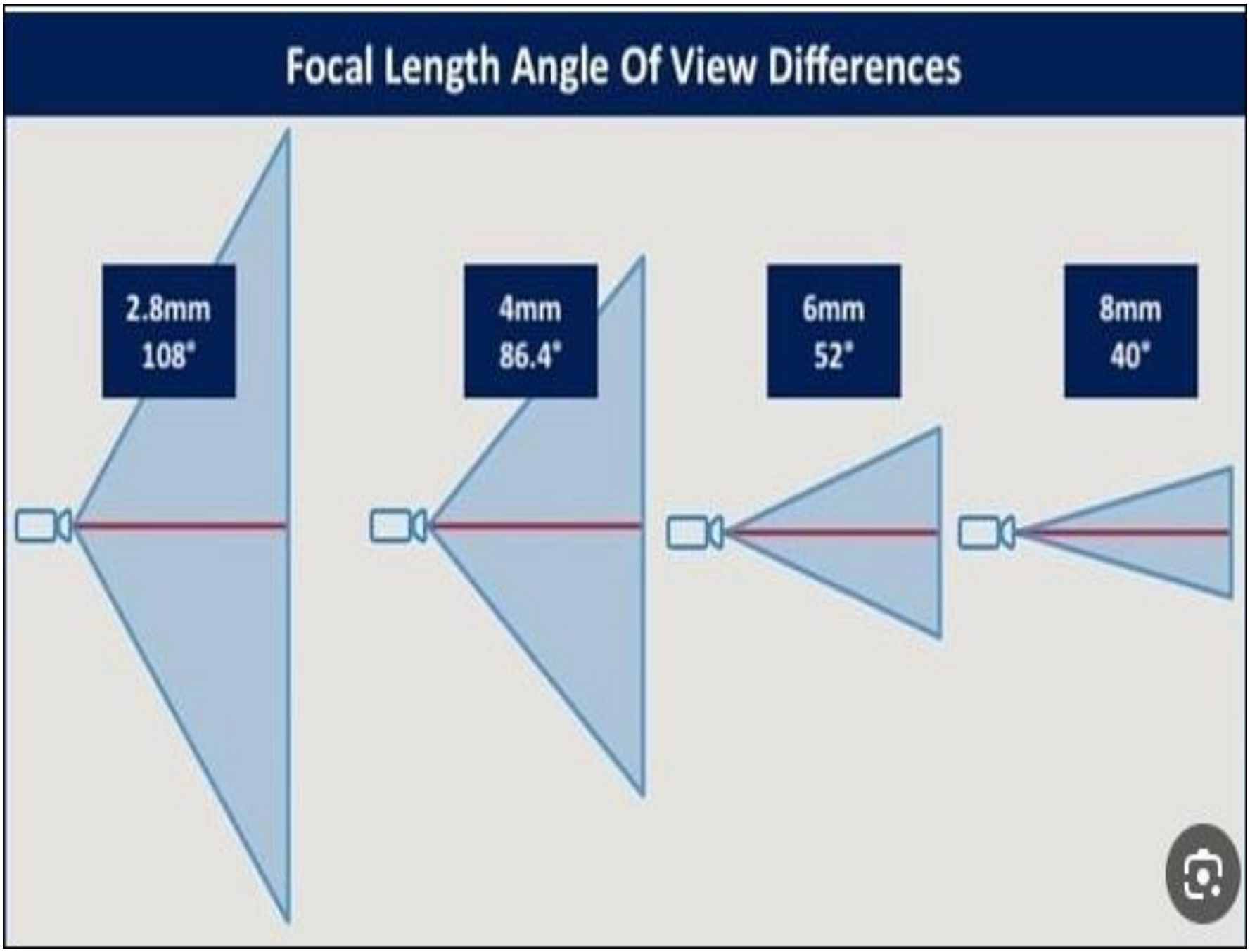


3- Angle view

- *Height angle:* determines how much of the scene we will fit into our view volume; larger height angles fit more of the scene into the view volume

– the greater the angle, the greater the amount of perspective distortion



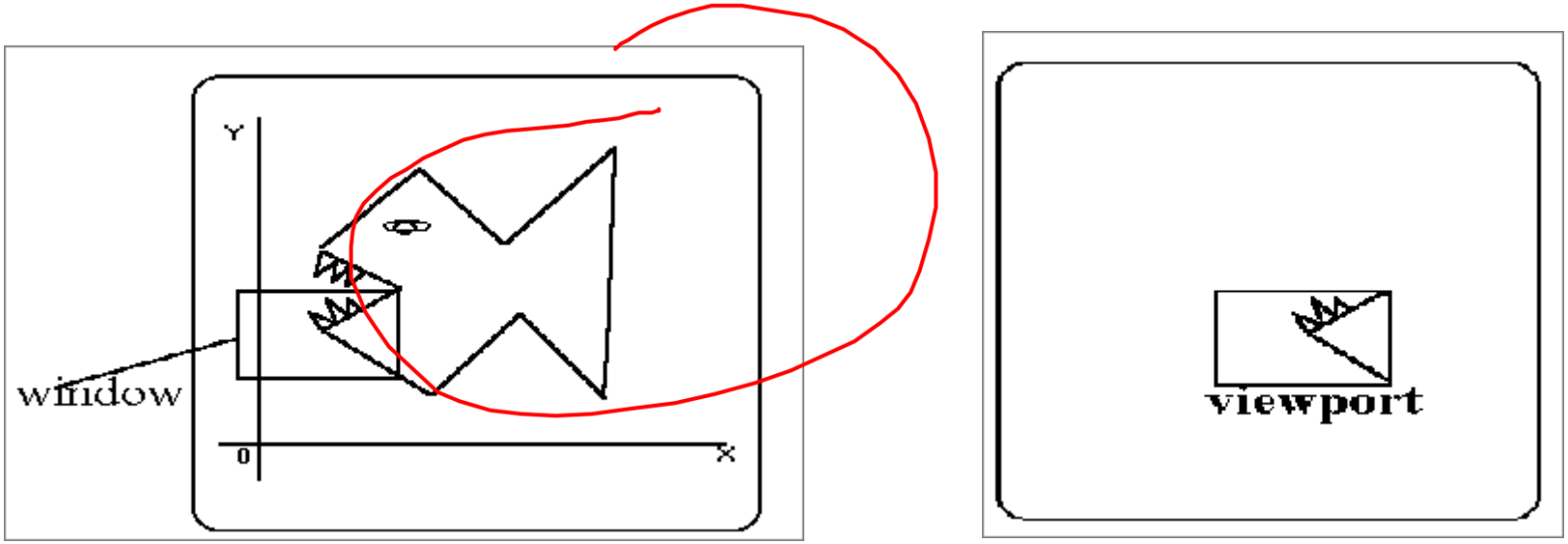

Focal Length Angle Of View Differences
2.8mm
108°
4mm
86.4°
6mm
52°
8mm
40°

Aperture


CAPTURING DIFFERENT AMOUNTS OF LIGHT
NO LIFEGUARD ON DUTY
NO LIFEGUARD ON DUTY
NO LIFEGUARD ON DUTY
Large aperture
Medium aperture
Small aperture

Viewing and clipping

- Example



Viewing and Clipping

- **The viewing** is the position of points relative to the viewing

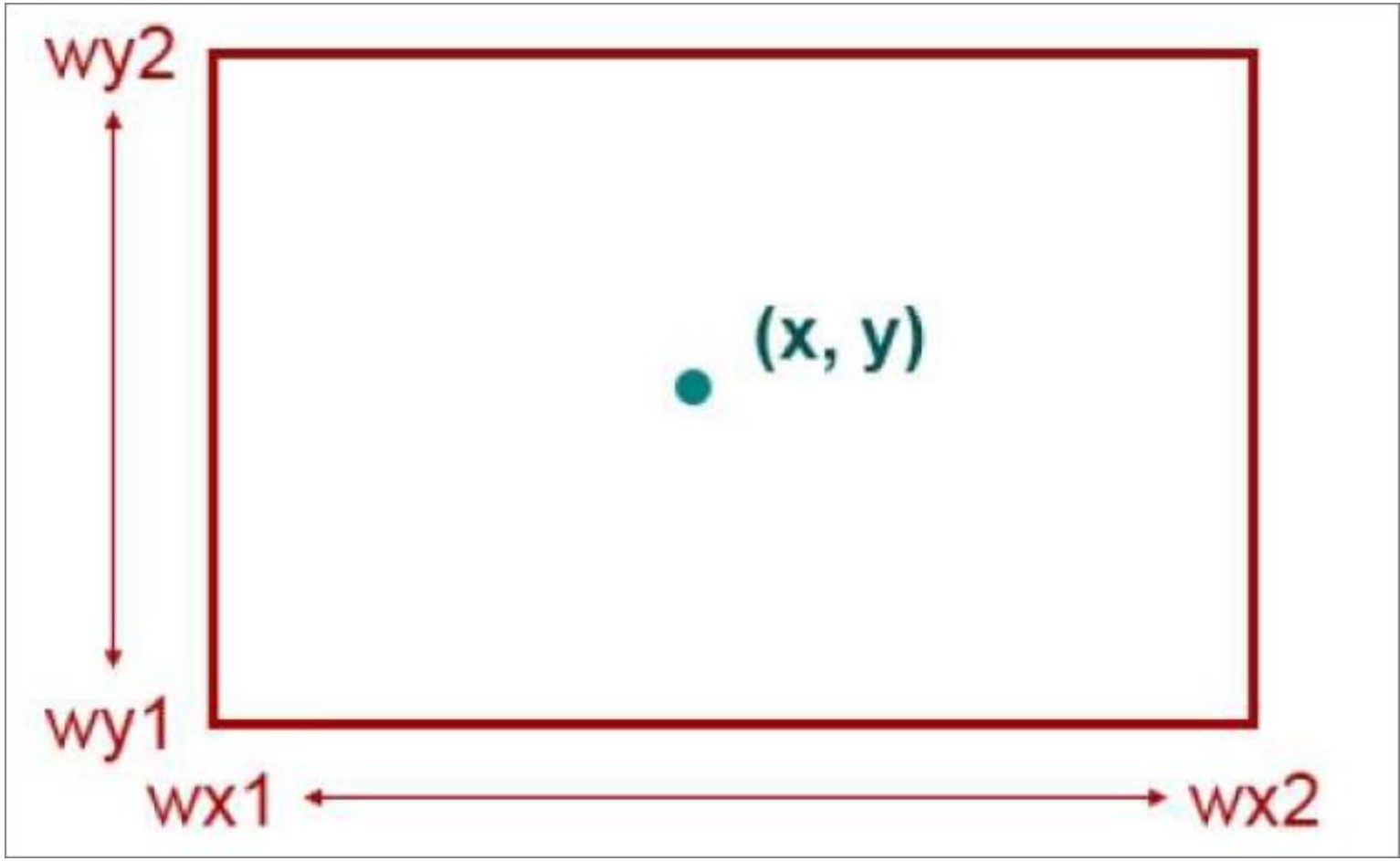
- **The Clipping** is removing points behind

the viewer – before generating the view.

(removing segments that are outside the viewing pane.)

1- Point Clipping

- Clipping a point from a given window is very easy.
- Point clipping tells us whether the given point X, Y is within the given window or not;
- and decides whether we will use the minimum and maximum coordinates of the window.



- The point is inside the window,
  - if X lies in between Wx1 ≤ X ≤ Wx2.
- And,
  - if Y lies in between Wy1 ≤ Y ≤ Wy2.

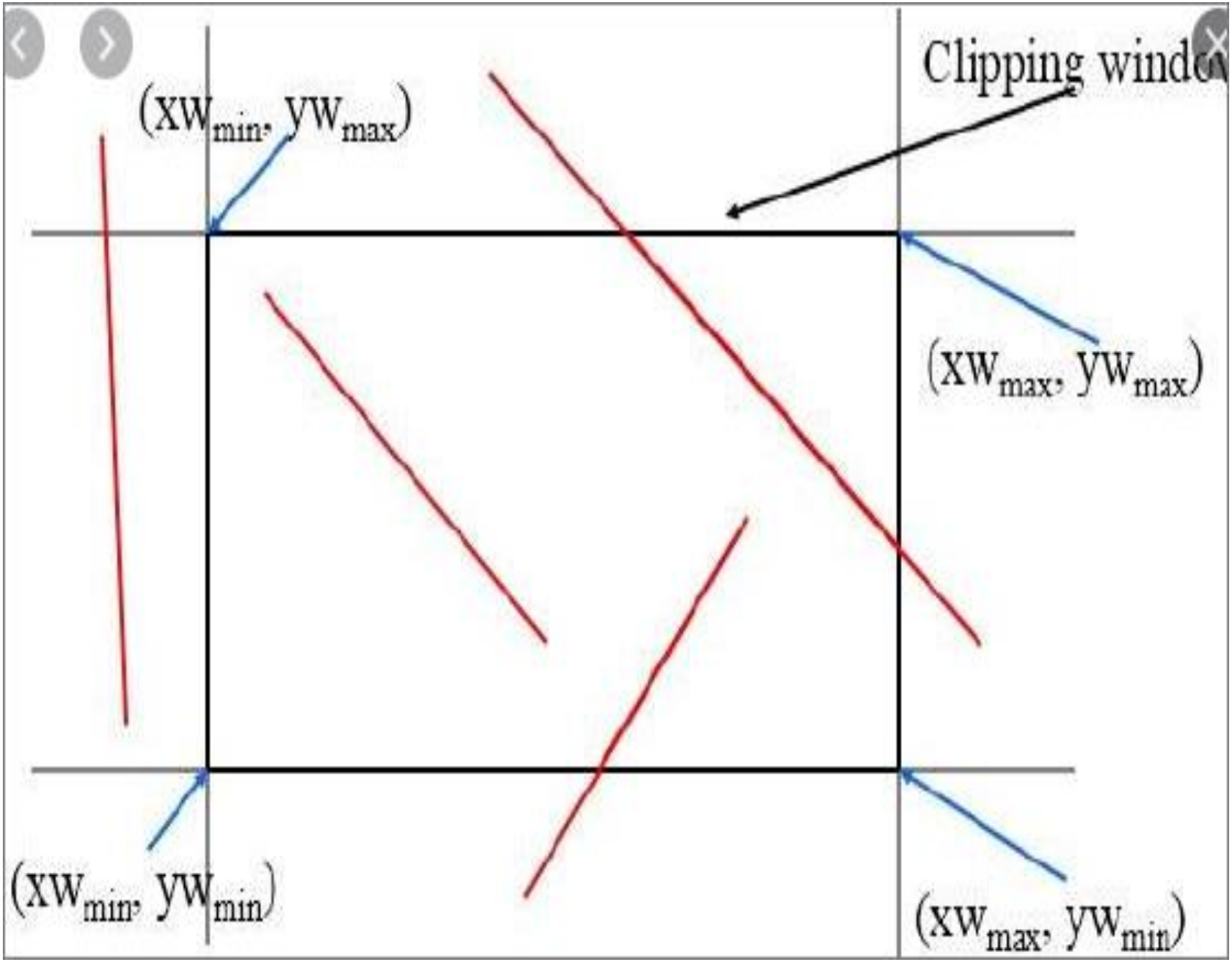
2- Line Clipping

- The concept of line clipping is same as point clipping.
- In line clipping, we will cut the portion of line which is outside of window and keep only the portion that is inside the window.

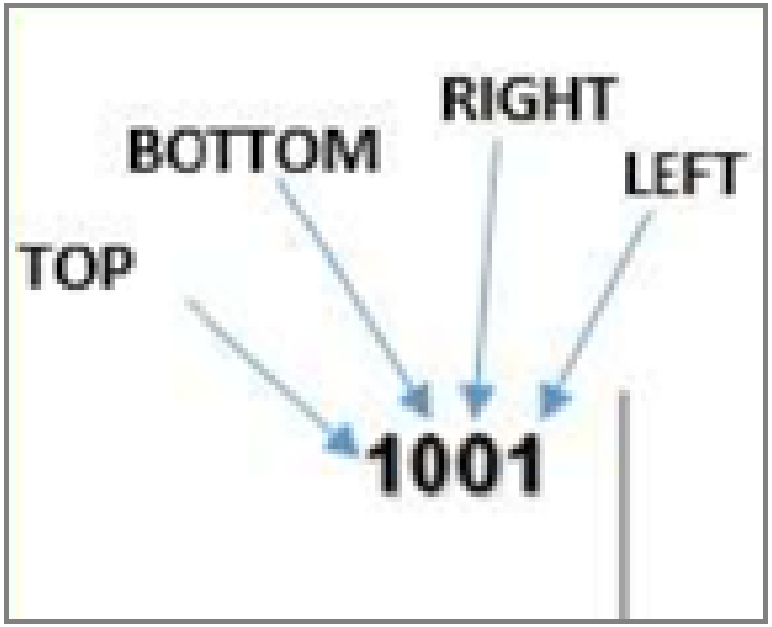
Method 1- Cohen- Line Clippings

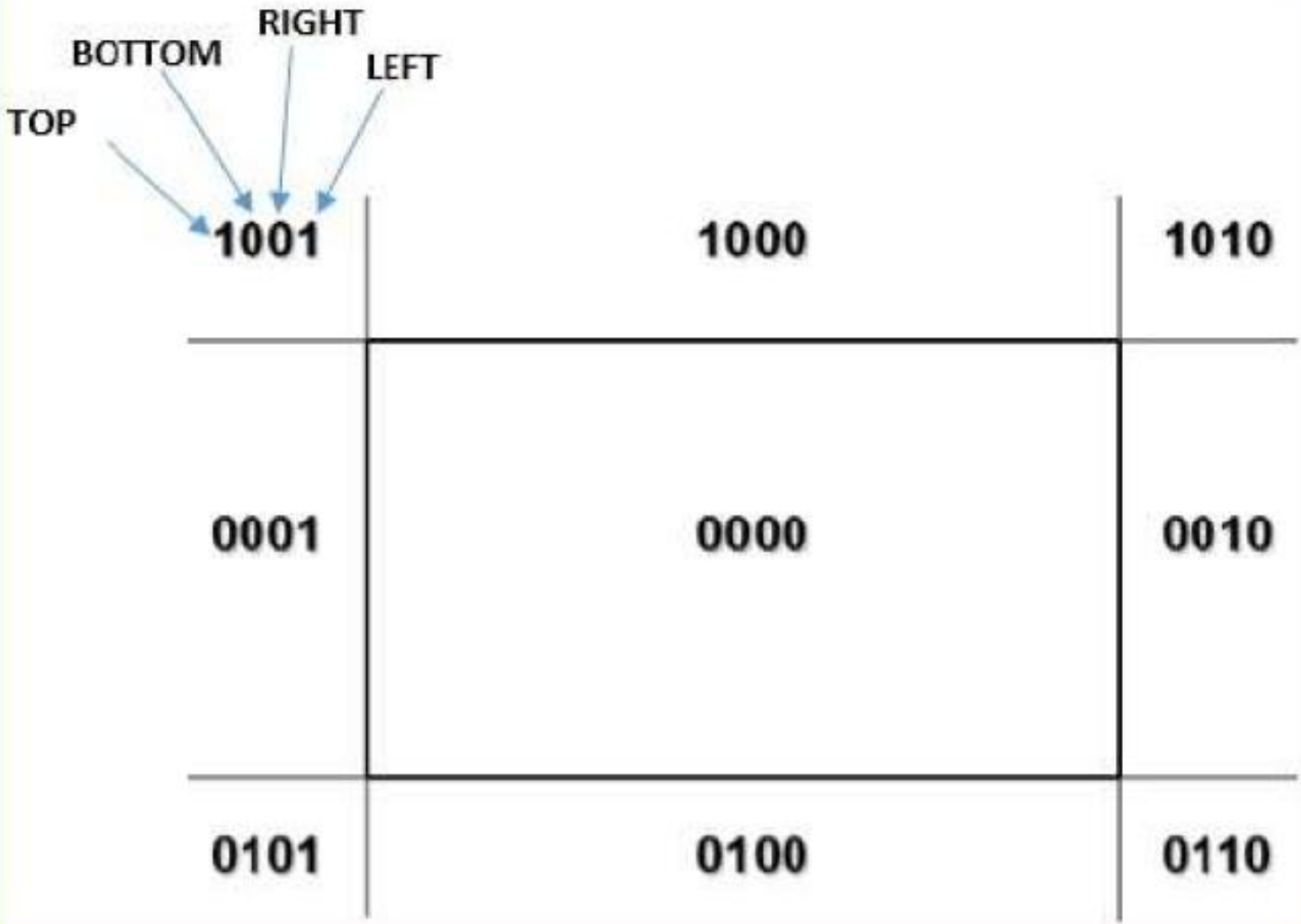
Line clipping:
Cohen- Line Clippings

- This algorithm uses the clipping window as:
- for the clipping region (window) is
  - (*XWmin* , *YWmin*)
  - (*XWmax*, *YWmax*).


Clipping window
(xw_{min}, yw_{max})
(xw_{max}, yw_{max})
(xw_{min}, yw_{min})
(xw_{max}, yw_{min})

- We will use 4-bits to divide the entire region. These 4 bits represent the
- Top,
- Bottom,
- Right, and
- Left of the region.




RIGHT
BOTTOM
LEFT
TOP
1001
1000
1010
0001
0000
0010
0101
0100
0110

There are 3 possibilities for the line

- Line can be completely inside the window

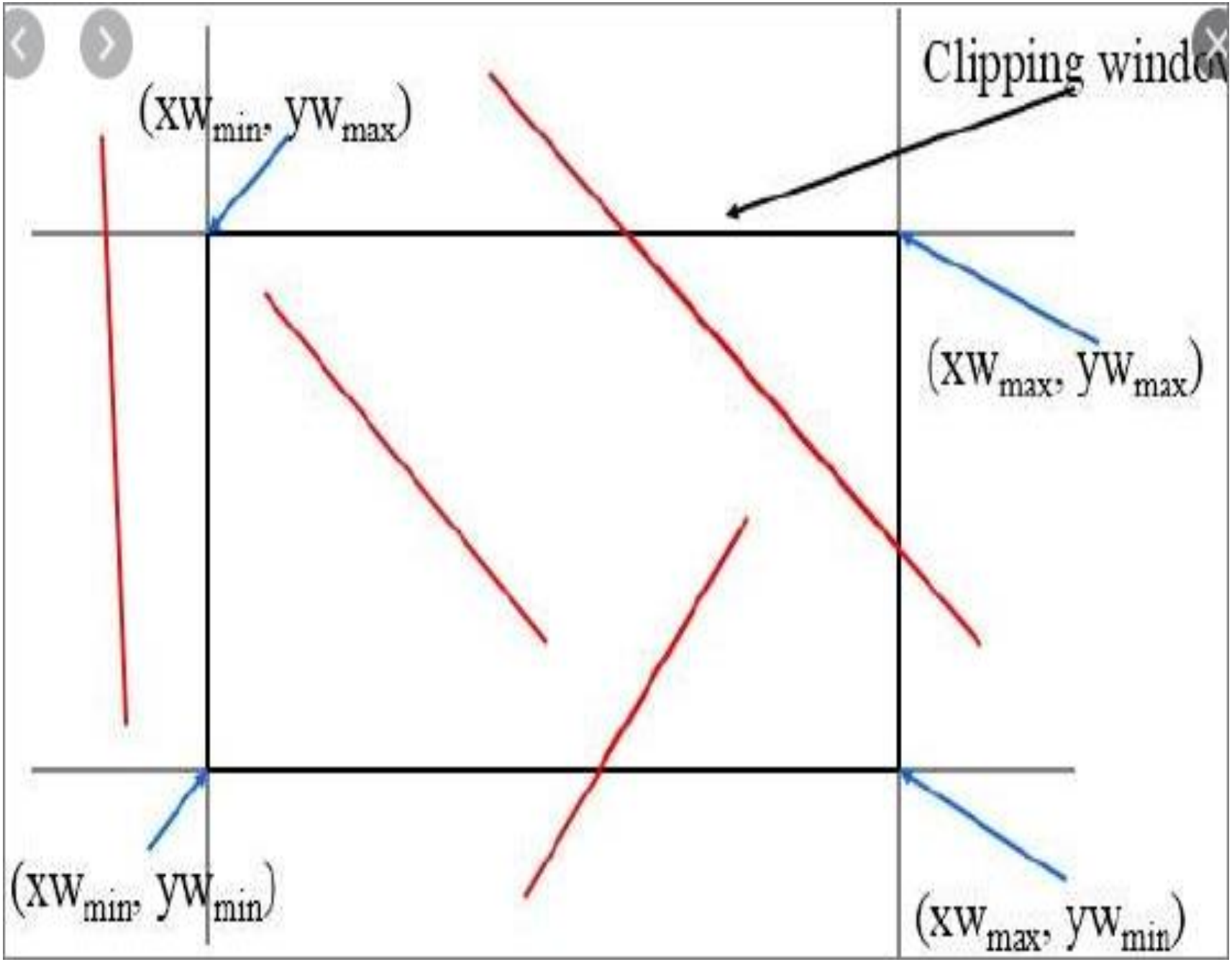
This line will be completely inside the region.

- Line can be completely outside of the window

This line will be completely removed from the region.

- Line can be partially inside the window

We will find intersection points and draw only that portion of line that is inside region.


Clipping window
(xw_{min}, yw_{max})
(xw_{max}, yw_{max})
(xw_{min}, yw_{min})
(xw_{max}, yw_{min})

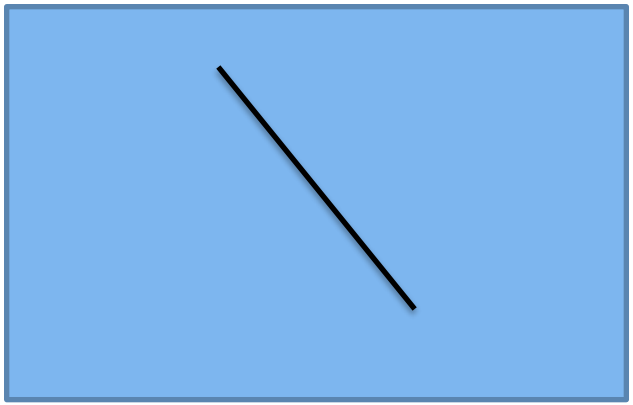
Algorithm

- **Step 1 −** Assign a region code for each endpoints.
- **Step 2 −** If both endpoints have a region code **0000** then accept this line.
- **Step 3 −** Else, perform the logical **AND** operation for both region codes.
  - **Step 3.1 −** If the result is not **0000,** then reject the line.
  - **Step 3.2 −** Else you need clipping.
    - **Step 3.2.1** − Choose an endpoint of the line that is outside the window.
    - **Step 3.2.2** − Find the intersection point at the window boundary *base on regioncode*.
    - **Step 3.2.3** − Replace endpoint with the intersection point and update the region code.
    - **Step 3.2.4** − Repeat step 2 until we find a clipped line either trivially accepted or trivially rejected.
- **Step 4 −** Repeat step 1 for other lines.

Case 1

- P1→0000 zero
- P2→0000 zero

And 0000 zero



Visible line

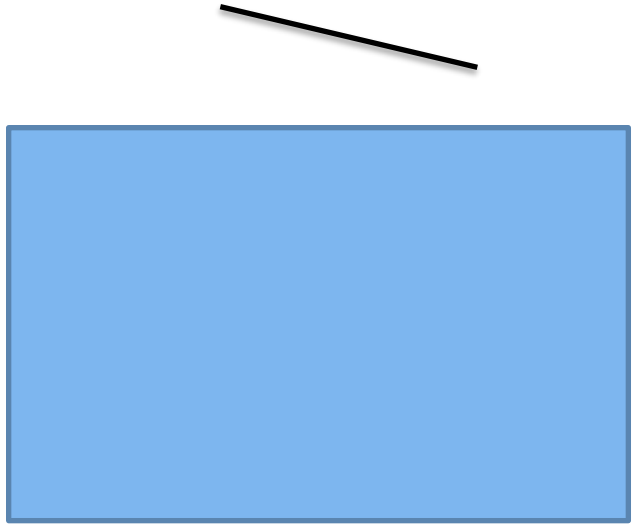
Accept line

No clipping

Case 2

- P3→1000 non zero
- P4→1000 non zero

And 1000 non zero



Not Visible line

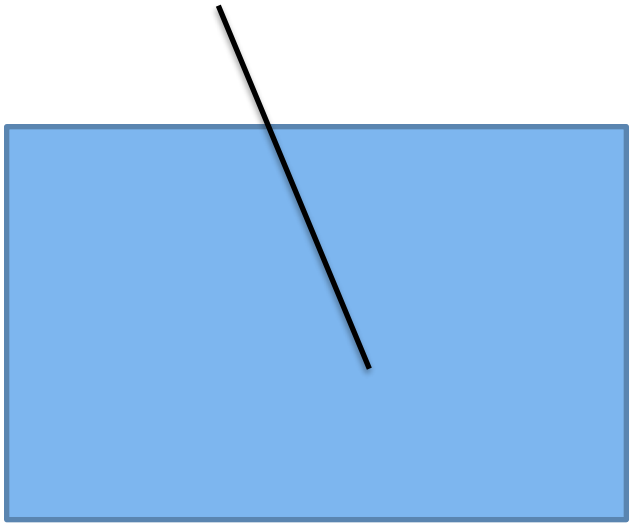
reject line

No clipping

Case 3

- P5→1000 non zero
- P6→0000 zero

And 0000 zero



Means that some part of line inside and some part outside

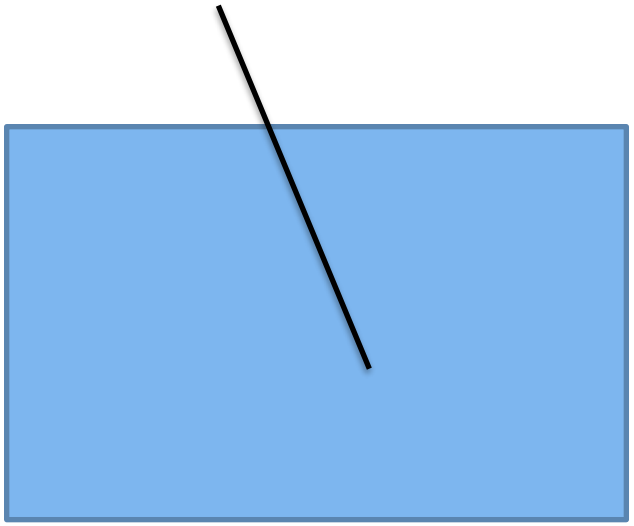
Clipping required

Case 3

- P5’→0000 zero
- P6→0000 zero

And 0000 zero

accept

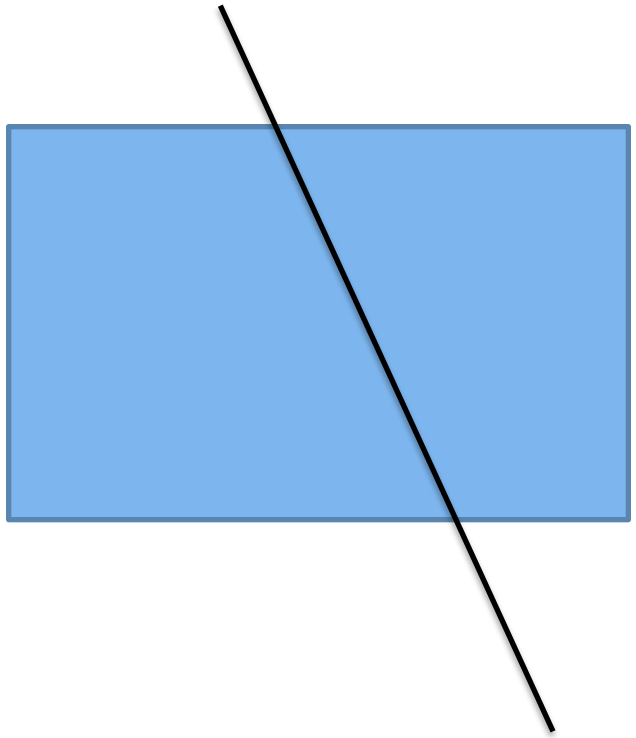


Case 4

- P7→1000 nonzero
- P8→0100 nonzero

And 0000 zero

Need clipping

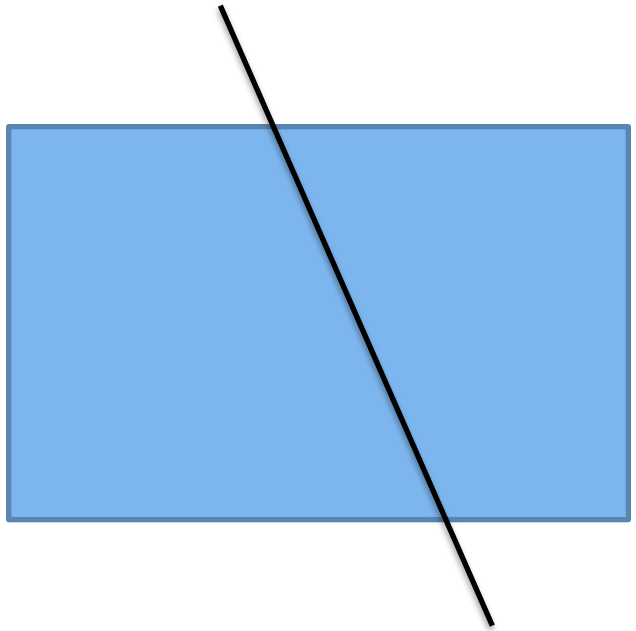


Case 4

- P7’→0000 zero
- P8→0100 nonzero

And 0000 zero

Need clipping

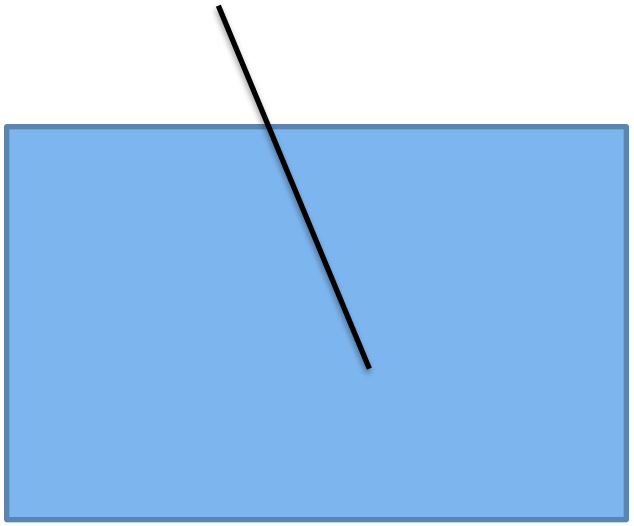


Case 4

- P7’→0000 zero
- P8’→0000 zero

And 0000 zero

accept



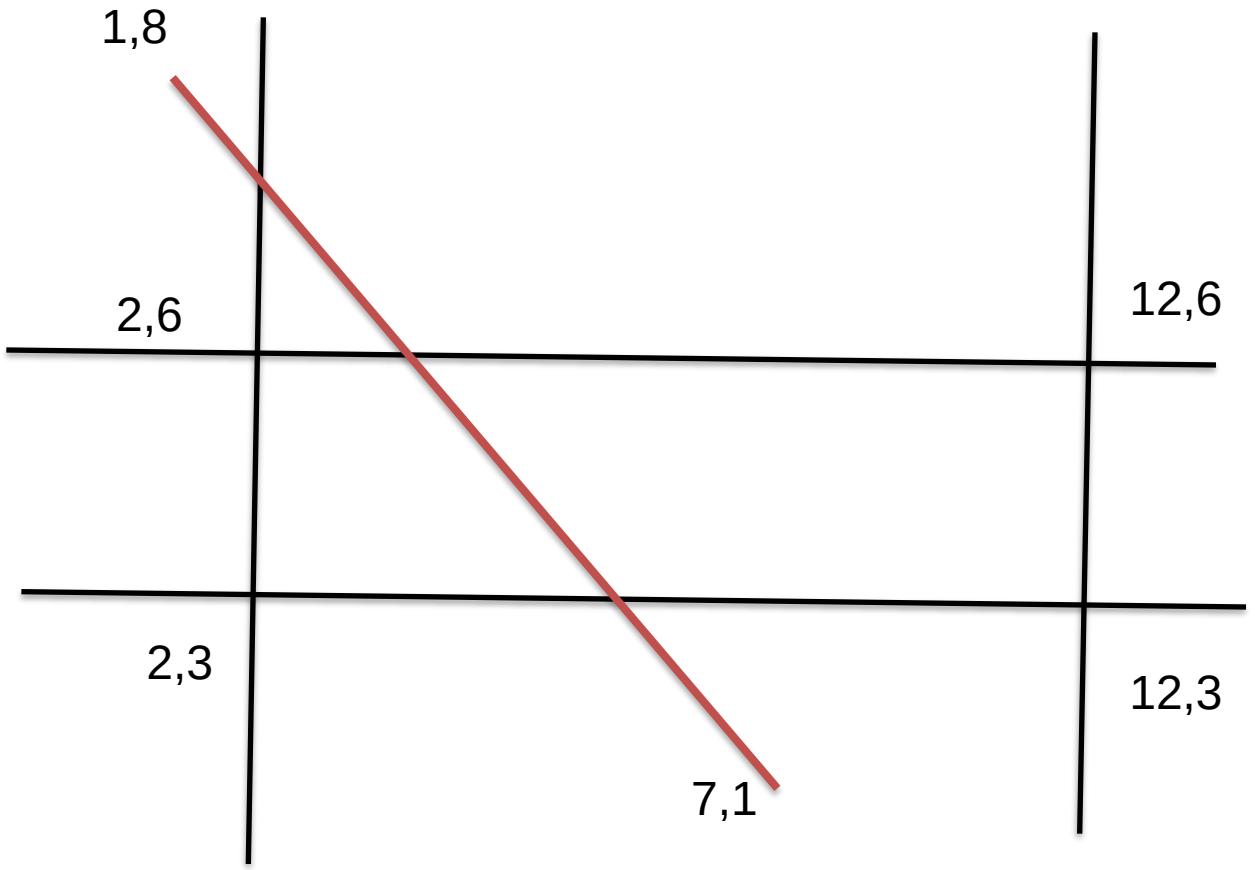
Intersection point

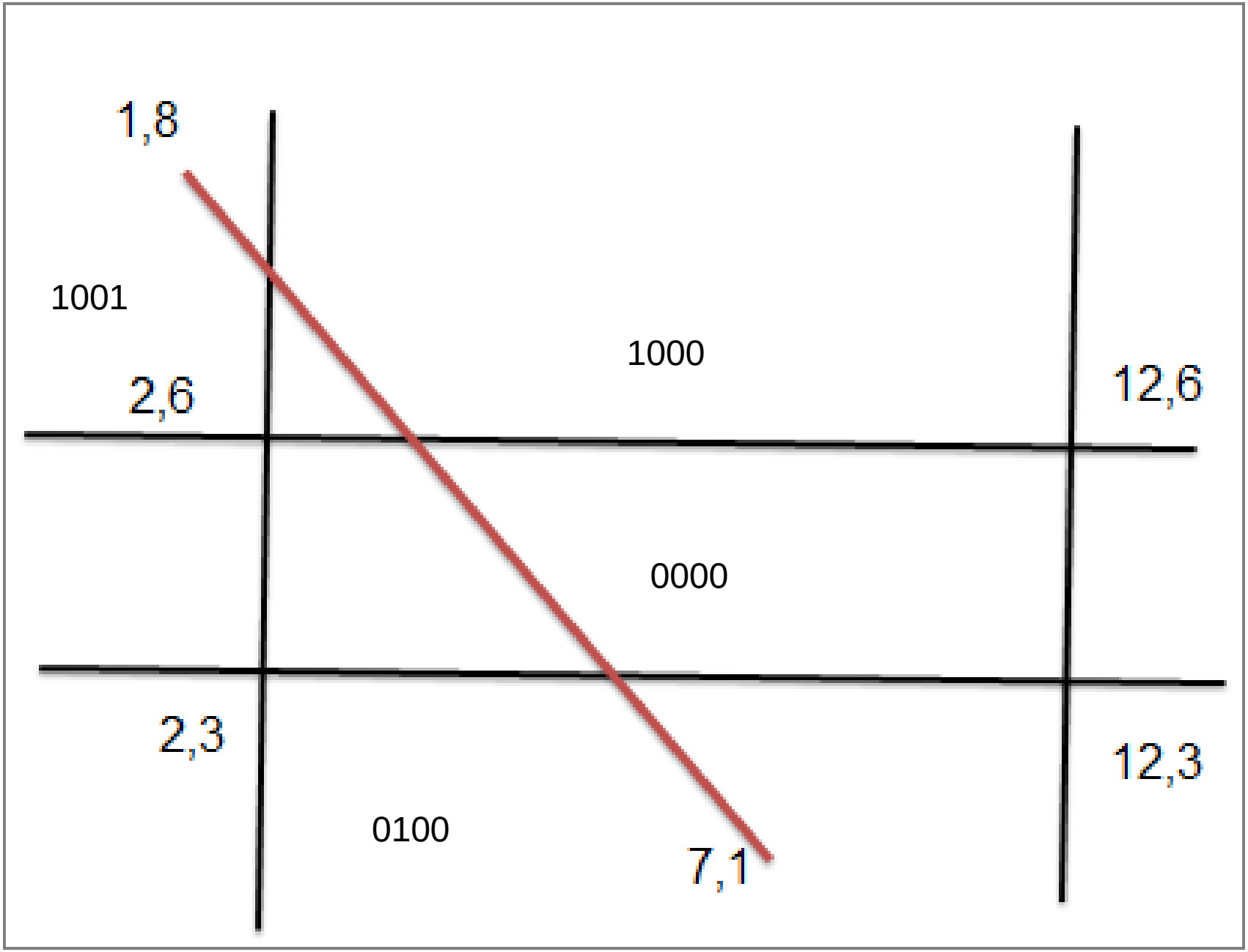
- Calculate intersection point
- Line end points (x1,y1) and (x2,y2)
- Y’ of intersection
  - Y’=y1+m(xboundary – x1)
  - Xboundary is x window max or min
- x’ of intersection
  - x’=x1+1/m(yboundary – y1)
  - yboundary is y window max or min

Example

- using Cohen line clipping algorithm to find intersection points with the window where the Window’s coordinates are: (2,3) , (12,3) , (2,6) and (12,6). The Line endpoints (1,8) and (7,1)

the Window’s coordinates are: (2,3) , (12,3) , (2,6) and (12,6).
The Line endpoints (1,8) and (7,1)




1,8
1001
1000
2,6
12,6
0000
2,3
12,3
0100
7,1

P1→1001

P2→ 0100

And 0000 clipping required

P1'→ 0000

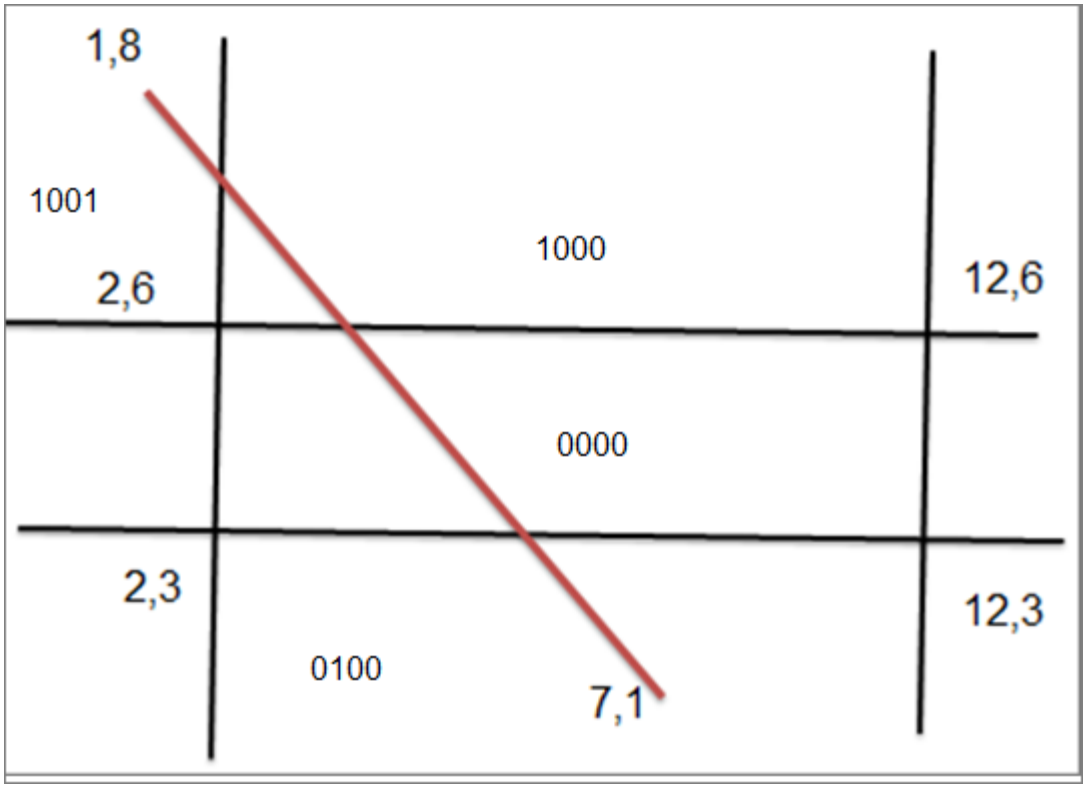
P2→ 0100

And 0000 clipping required

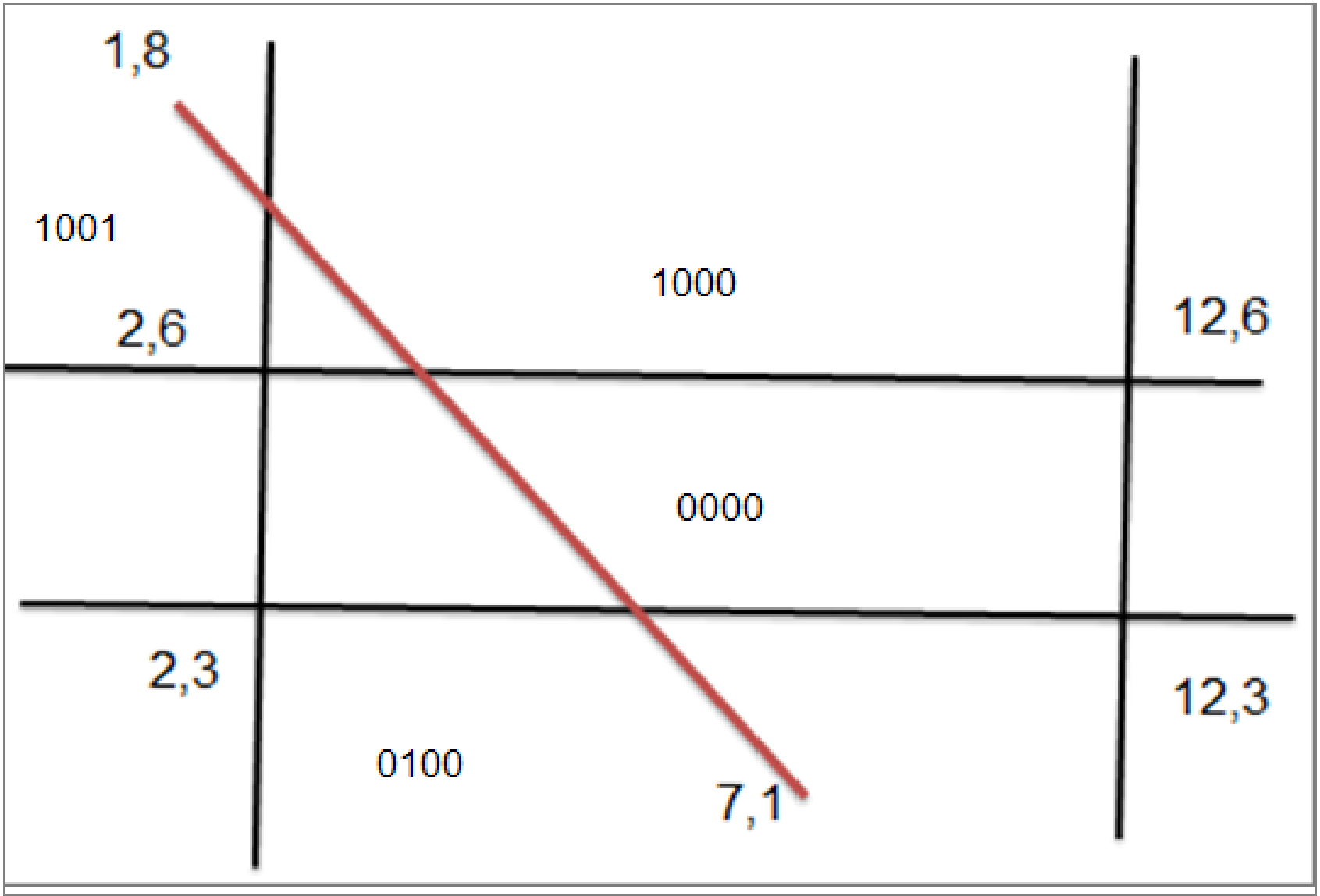
P1'→ 0000

P2'→ 0000

And 0000 accept



Intersection points



- y’ of intersection is $y' = y1 + m(x_{boundary} - x1)$

- x’ of intersection is $x' = x1 + 1/m(y_{boundary} - y1)$

where $X_{boundary}$ is x_{wmax} or x_{wmin}

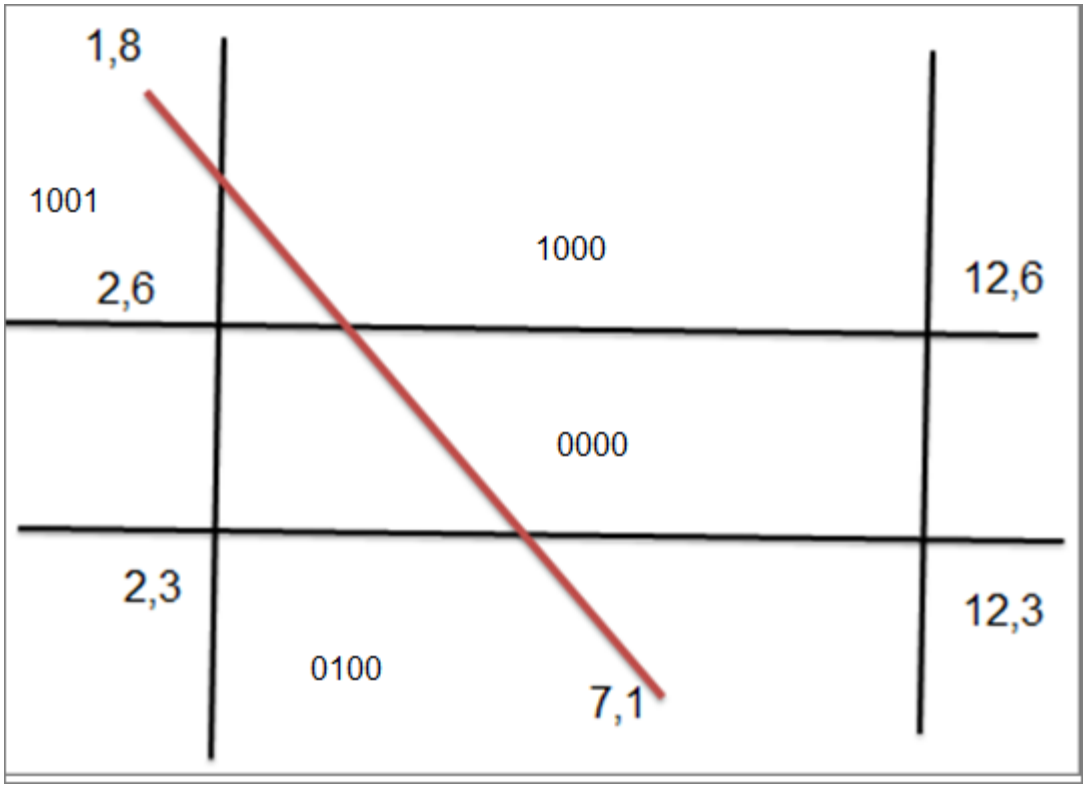
and

- $y_{boundary}$ is y_{wmax} or y_{wmin}

- if intersection in the top then $y' = y_{wmax}$
- if intersection in the bottom then $y' = y_{wmin}$
- if intersection in the left then $x' = x_{wmin}$
- if intersection in the right then $x' = x_{wmax}$

- First intersection (x',y')
- $Y' = y_{wmax}$ $X' = x1 + m(y_{wmax} - y1)$

- second intersection (x'',y'')
- $Y'' = y_{wmin}$ $X' = x2 + m(y_{wmin} - y2)$



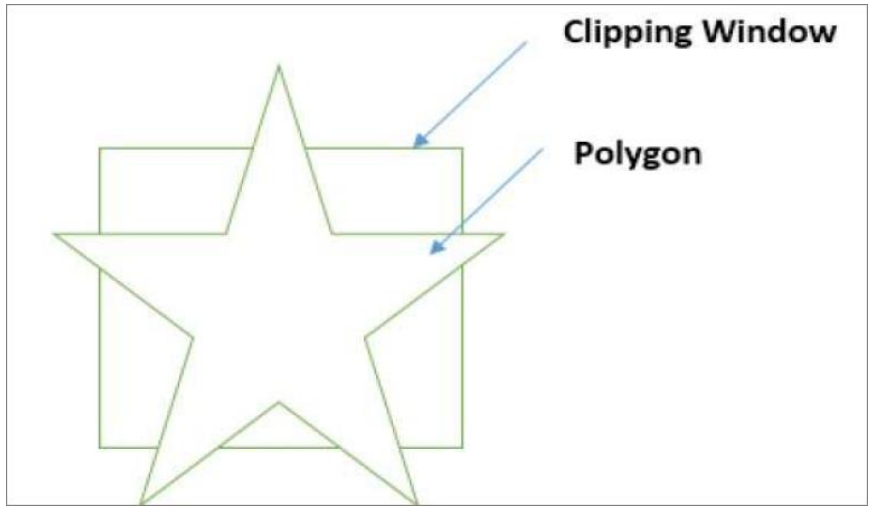
Clipping

- It is removing lines or portions of lines outside an area
  - Point
  - Line
    - Cohen– Algorithm
  - Polygon
    - Sutherland Hodgman Algorithm

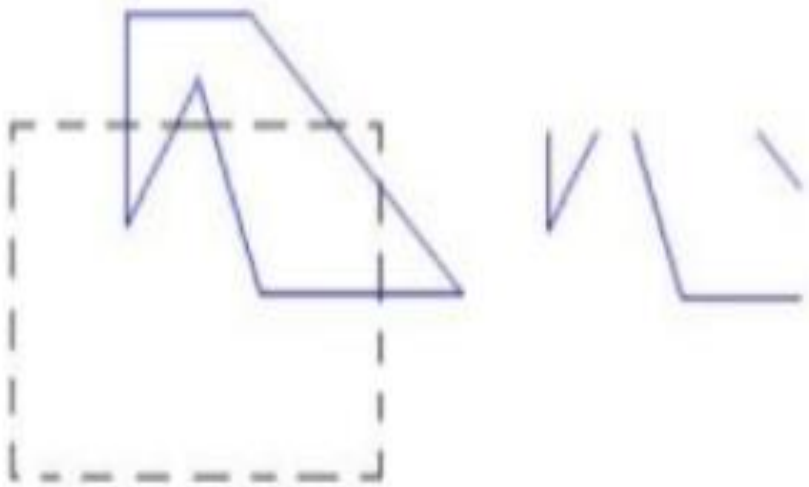
What is Polygon?

- Ordered set of vertices (points)
  - Usually counter-clockwise عکس
- Two consecutive vertices define an edge
- Last vertex implicitly connected to first

What is polygon clipping?



To clip a polygon, we cannot directly apply a line-clipping method to the individual polygon edges because this approach would produce a series of unconnected line segments as shown in figure .

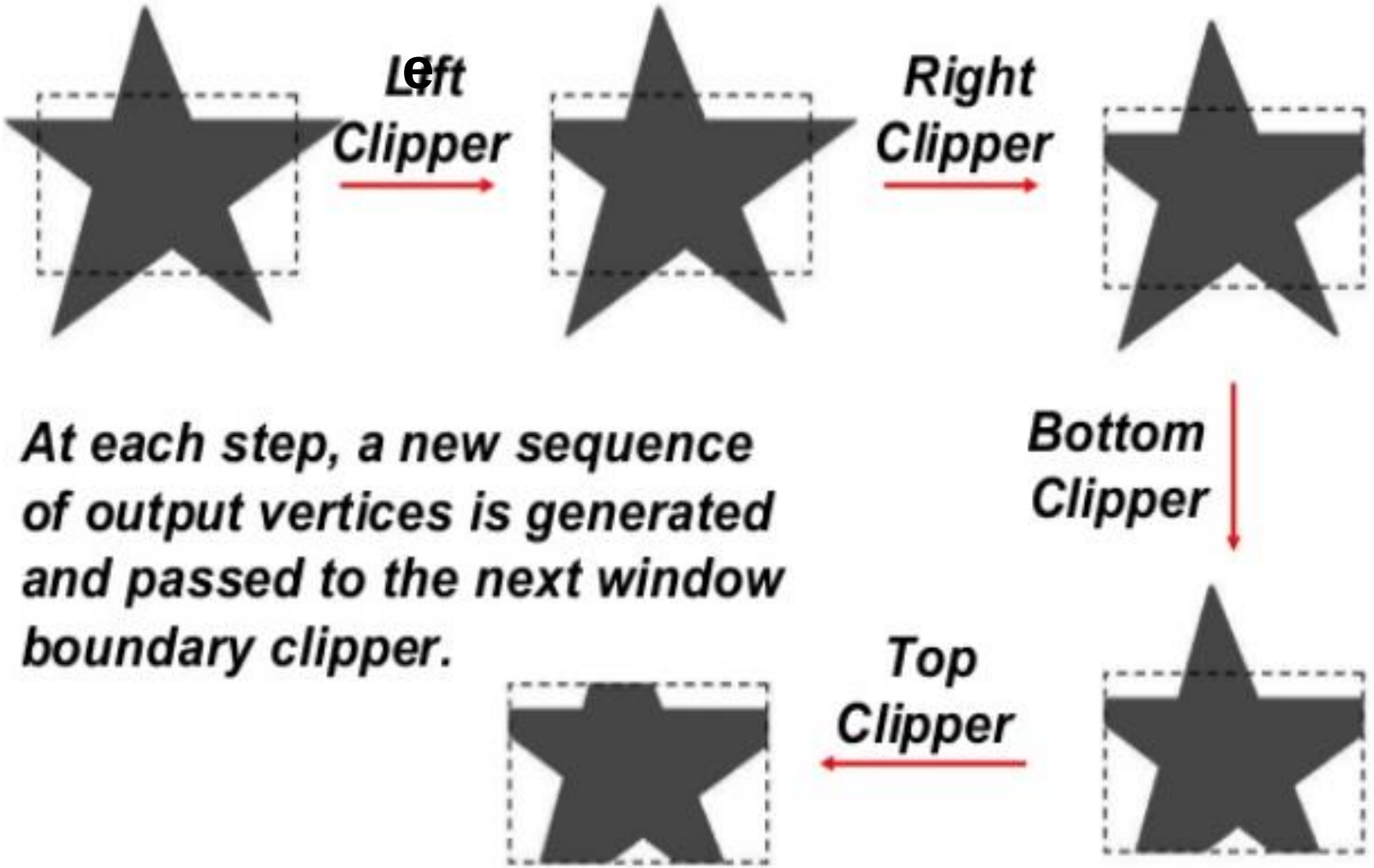


Polygon Clipping *Sutherland Hodgman Algorithm*

- In this algorithm, all the vertices of the polygon are clipped against each edge of the clipping window.

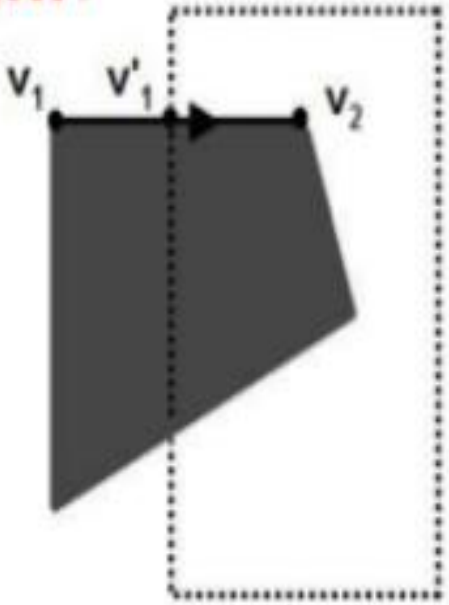
steps

- *Clip a polygon by processing the polygon boundary against each window edge. (Left, Top, Bottom, Right).*
- *Apply four cases to process all polygon edges to produce sequence of vertices. This sequence of vertices will make a bounded area.*
- *Beginning with the initial set of polygon vertices, we could first clip the polygon against the left rectangle boundary to produce a new sequence of vertices.*
- *The new set of vertices could be successively passed to a right boundary clipper, a bottom boundary clipper, and a top boundary clipper.*


Left
Clipper
Right
Clipper
Bottom
Clipper
Top
Clipper
At each step, a new sequence of output vertices is generated and passed to the next window boundary clipper.

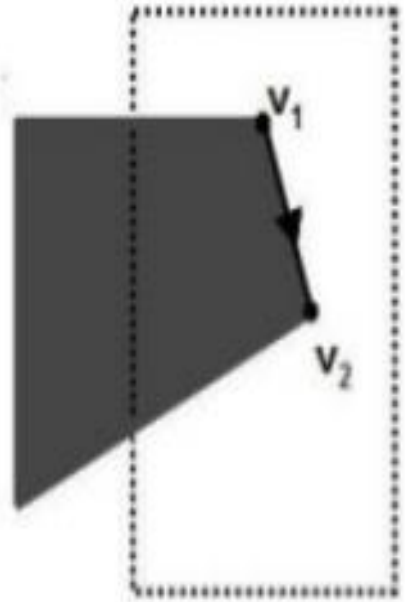
1. If the first vertex is outside the window boundary and the second vertex is inside

Then , both the intersection point of the polygon edge with the window boundary and the second vertex are added to the output vertex list



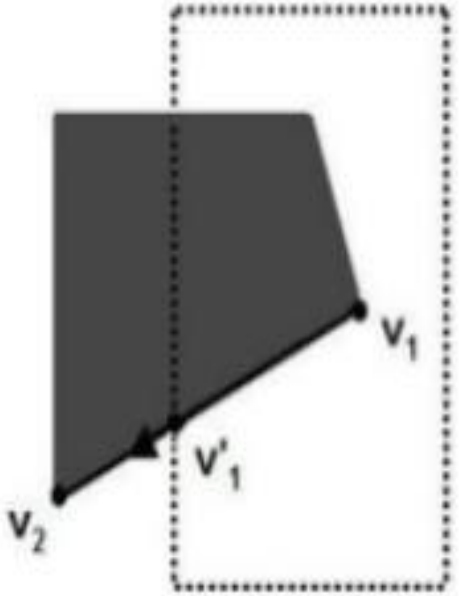
2. If both input vertices are inside the window boundary.

Then, only the second vertex is added to the output vertex list.



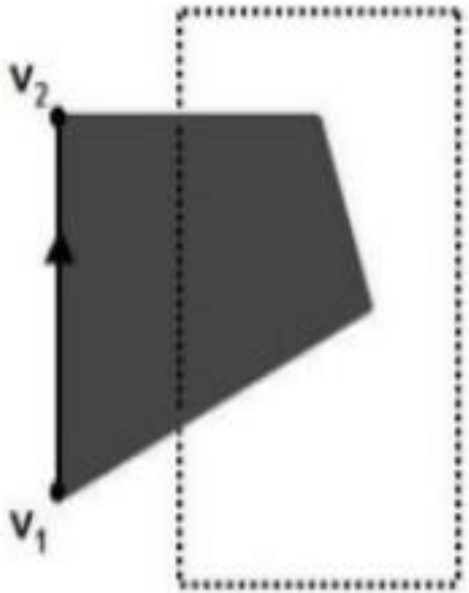
3. If the first vertex is inside the window boundary and the second vertex is outside.

Then, only the edge intersection with the window boundary is added to the output vertex list.



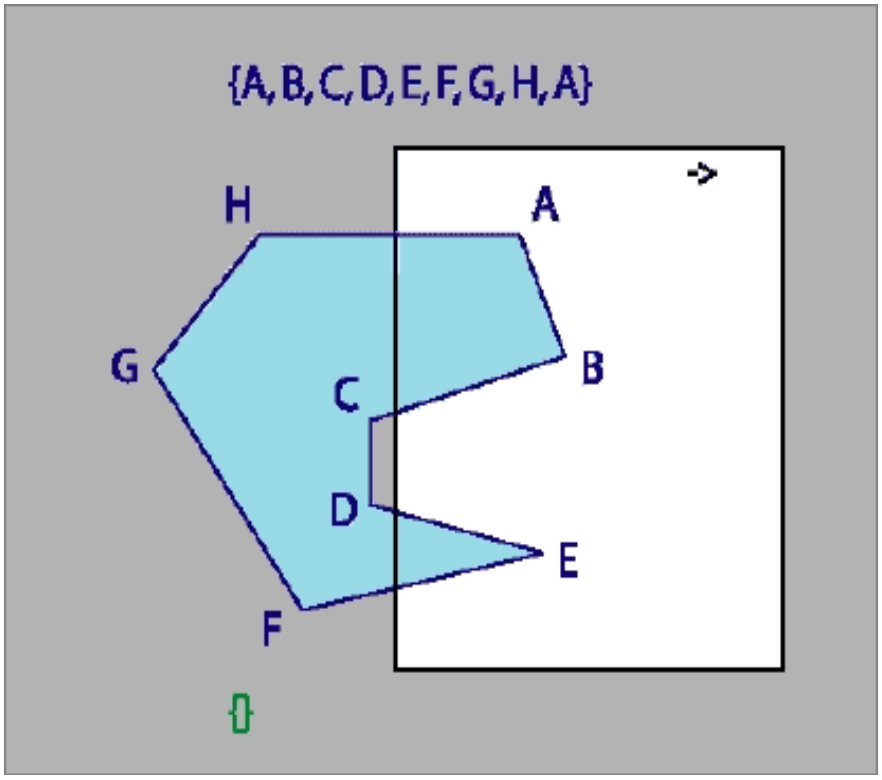
4. If both input vertices are outside the window boundary.

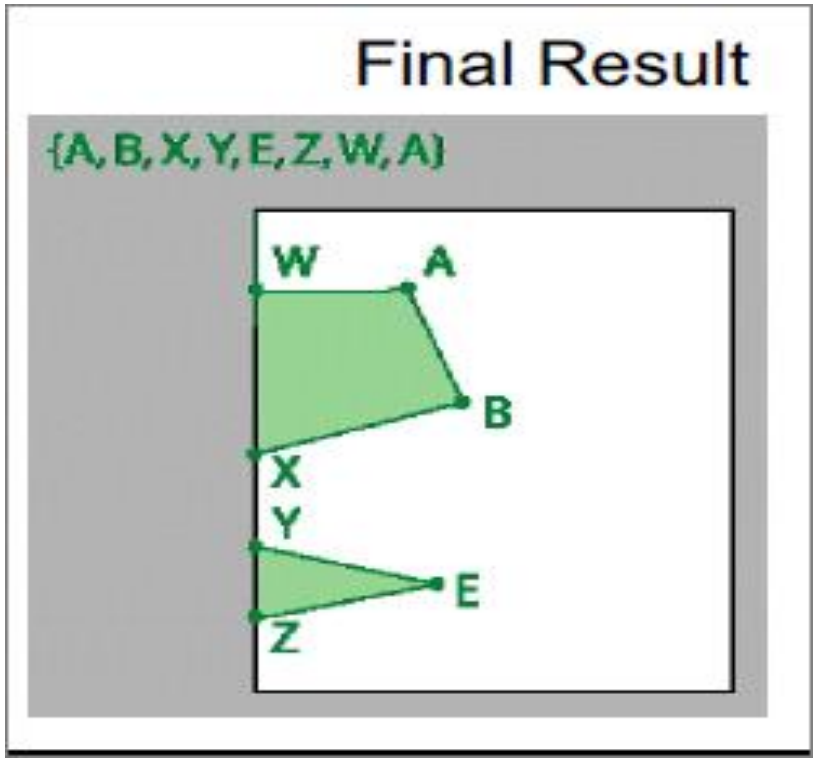
Then, nothing is added to the output vertex list.



idea

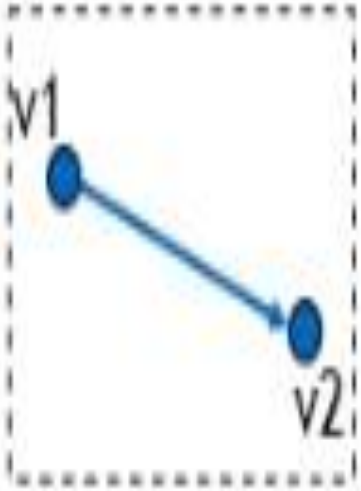
- Move around polygon from *v1* to *vn* and back to *v1*
- Input:
  - *v1, v2, … vn* the vertices defining the polygon
  - Single infinite clip edge w/ inside/outside info
- Output:
  - *v'1, v'2, … v'm*, vertices of the clipped polygon
  - Do this 4 times
  - Traverse vertices (edges)
  - Add vertices one-at-a-time to output polygon
  - Use inside/outside info
  - Edge intersections


{A,B,C,D,E,F,G,H,A}
H
A
G
B
C
D
E
F
{}

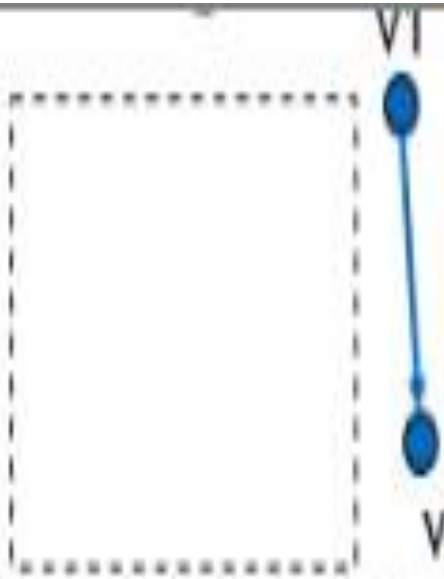

Final Result
{A,B,X,Y,E,Z,W,A}
W
A
B
X
Y
E
Z

❖Edgewise clipping

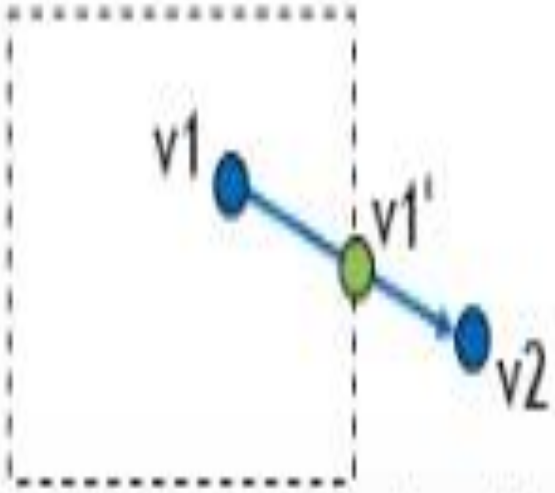
In order to clip polygon edges against a window edge we move from vertex V_i to the next vertex V_{i+1} and decide the output vertex according to following four simple cases:



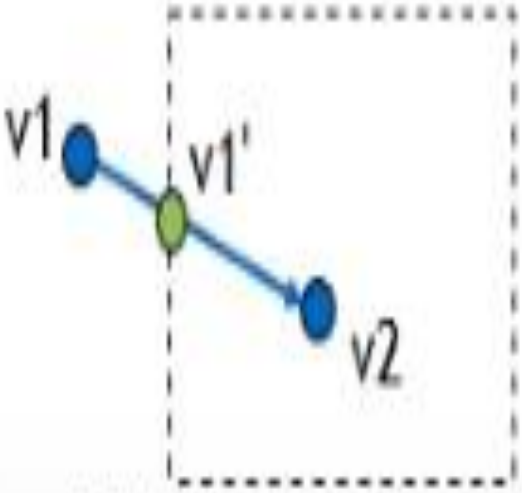
inside-inside -> save only v2



outside-outside -> save nothing

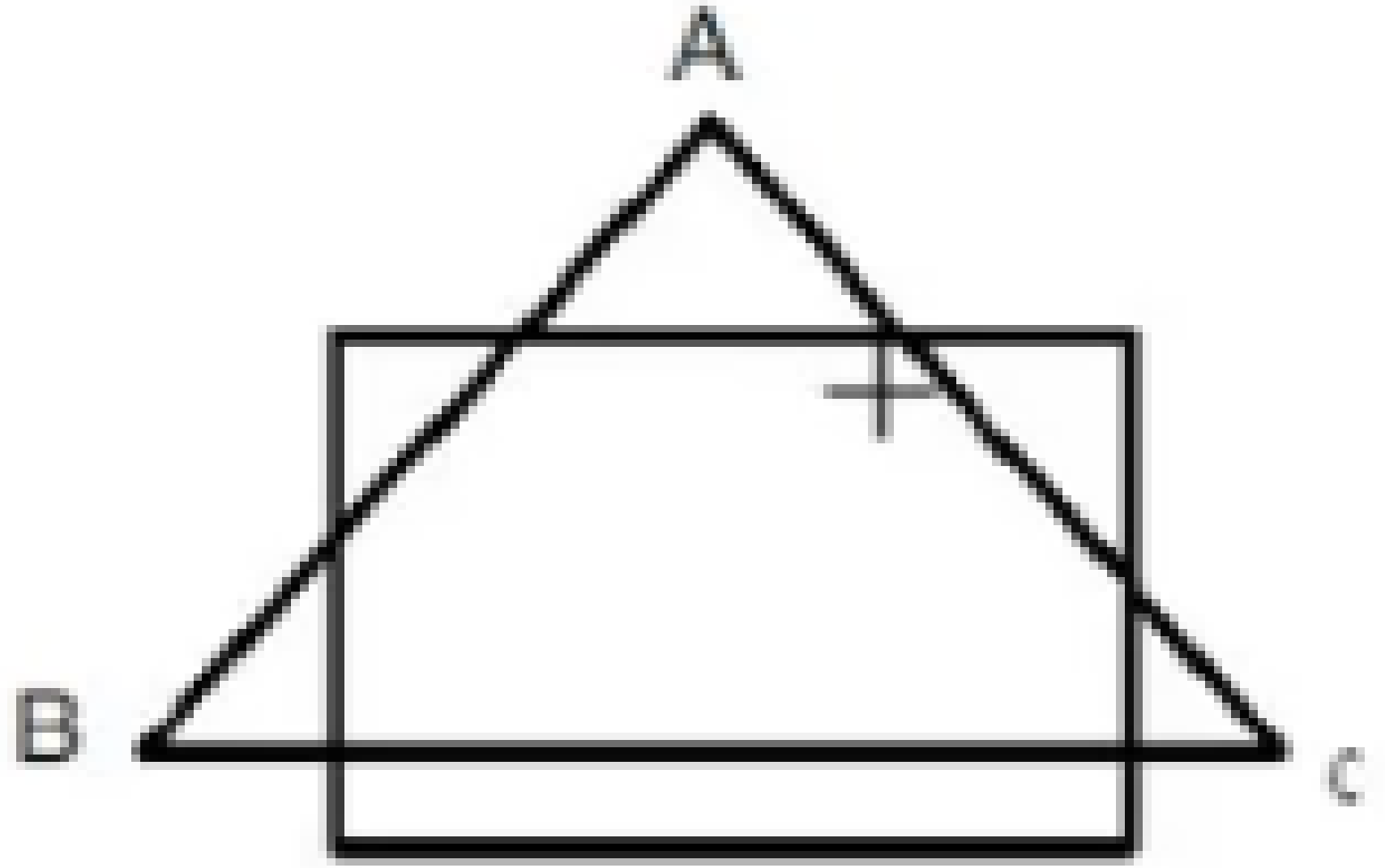


inside-outside -> save only v1'



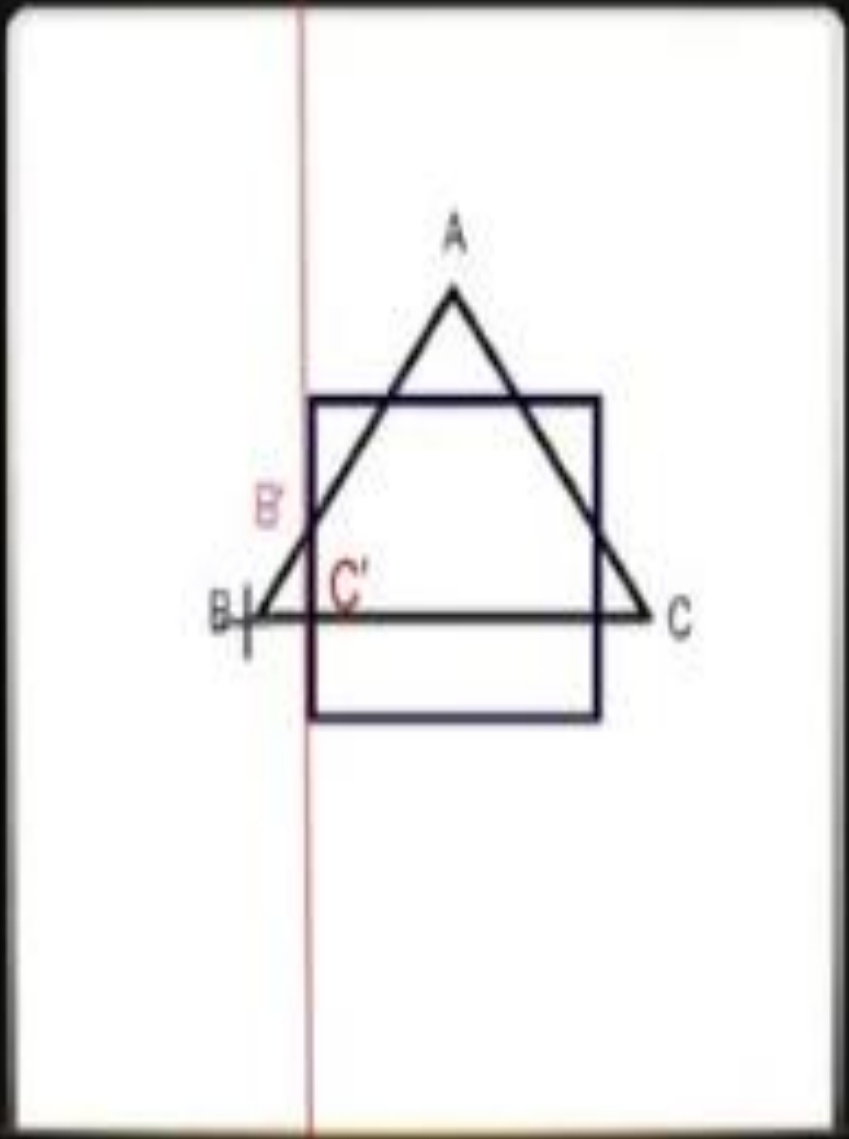
outside-inside -> save {v1', v2}

Example 1


A
B
C

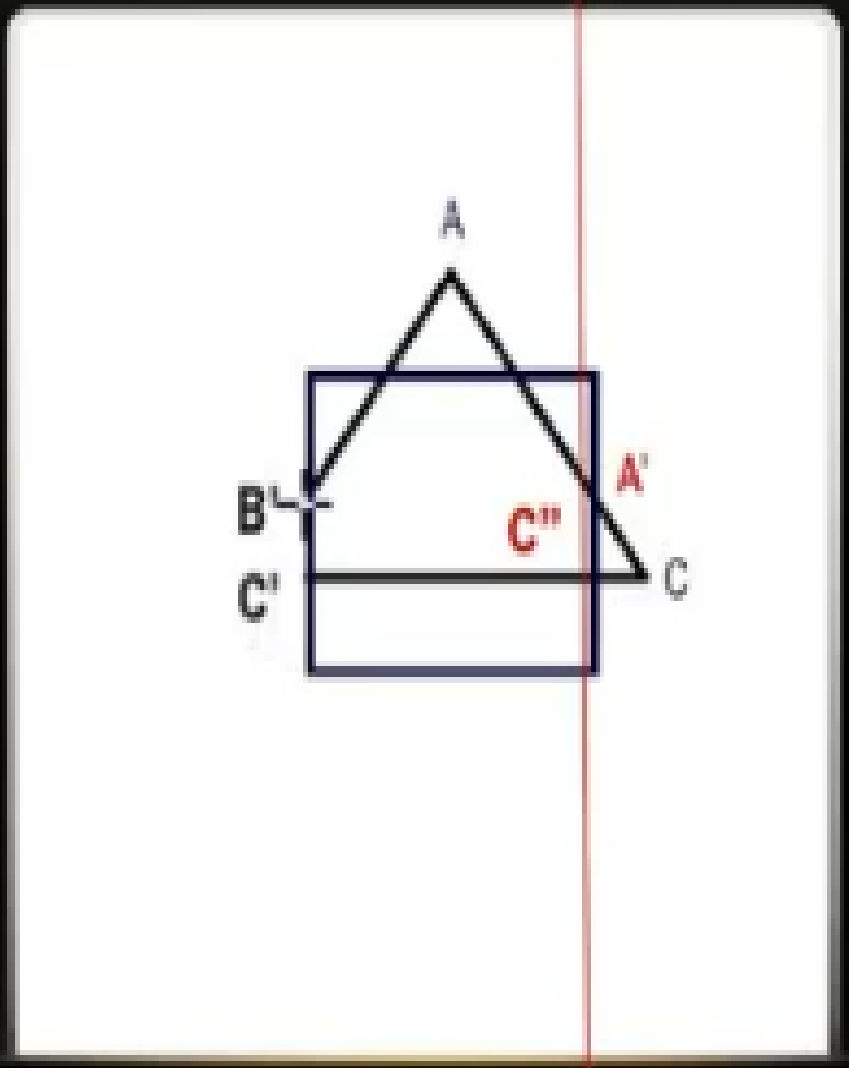
For Left

Edge	Action	Output
BA	OUT-IN	B',A
AC	IN-IN	C
CB	IN-OUT	C'



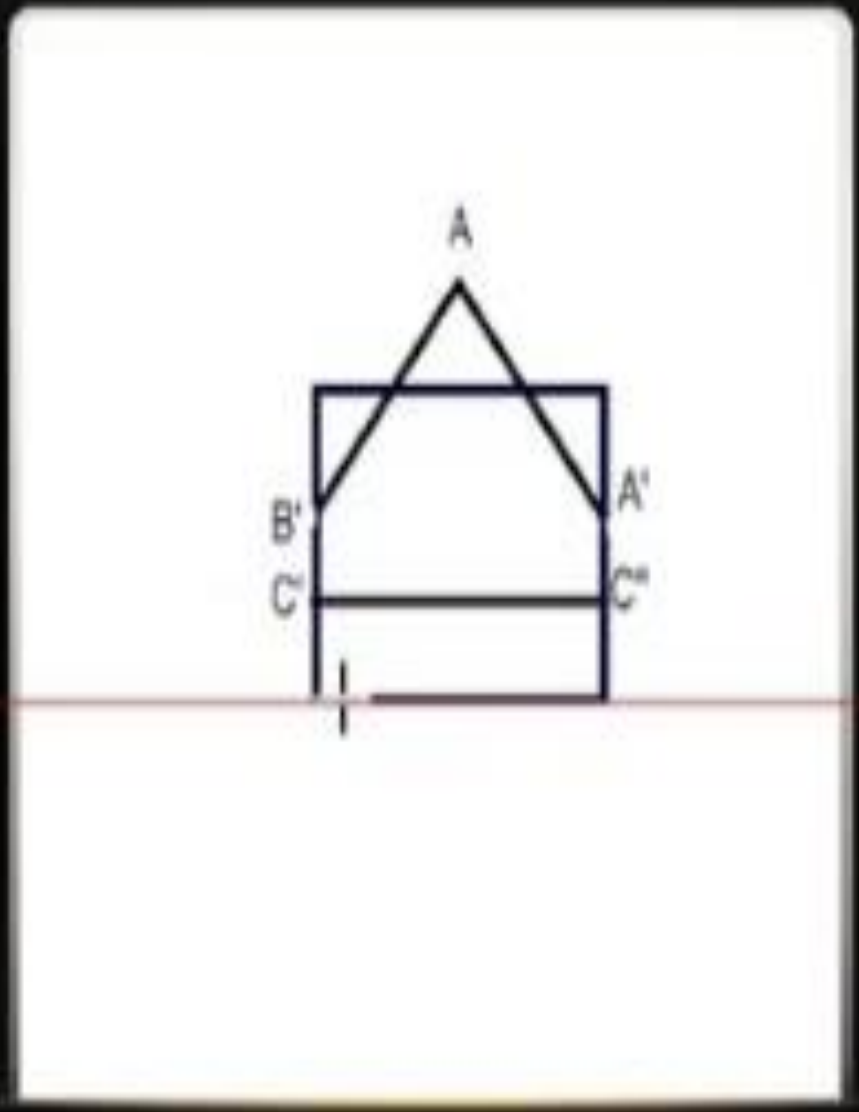
For Right

Edge	Action	Output
B'A	IN-IN	A
AC	IN-OUT	A'
CC'	IOUT-IN	C''C'
C'B'	IN-IN	B'



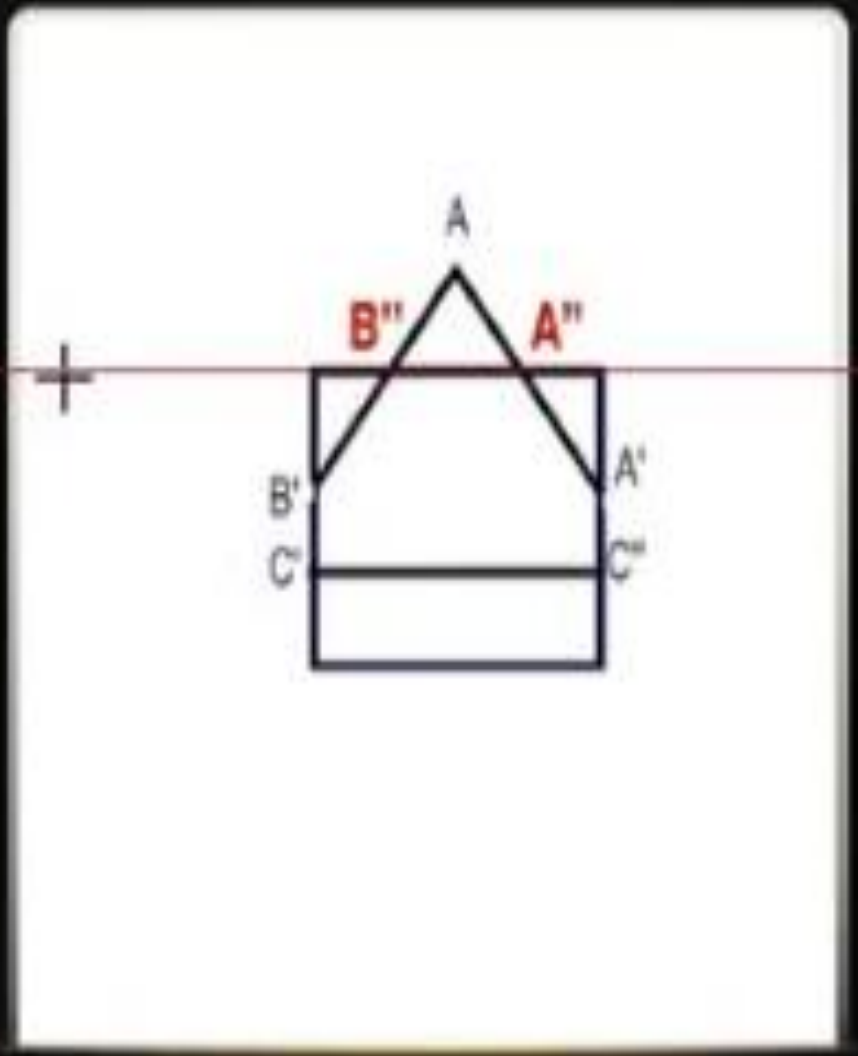
For Bottom

Edge	Action	Output
B'A	IN-IN	A
AA'	IN-IN	A'
A'C''	IN-IN	C''
C''C'	IN-IN	C'
C'B'	IN-IN	B'



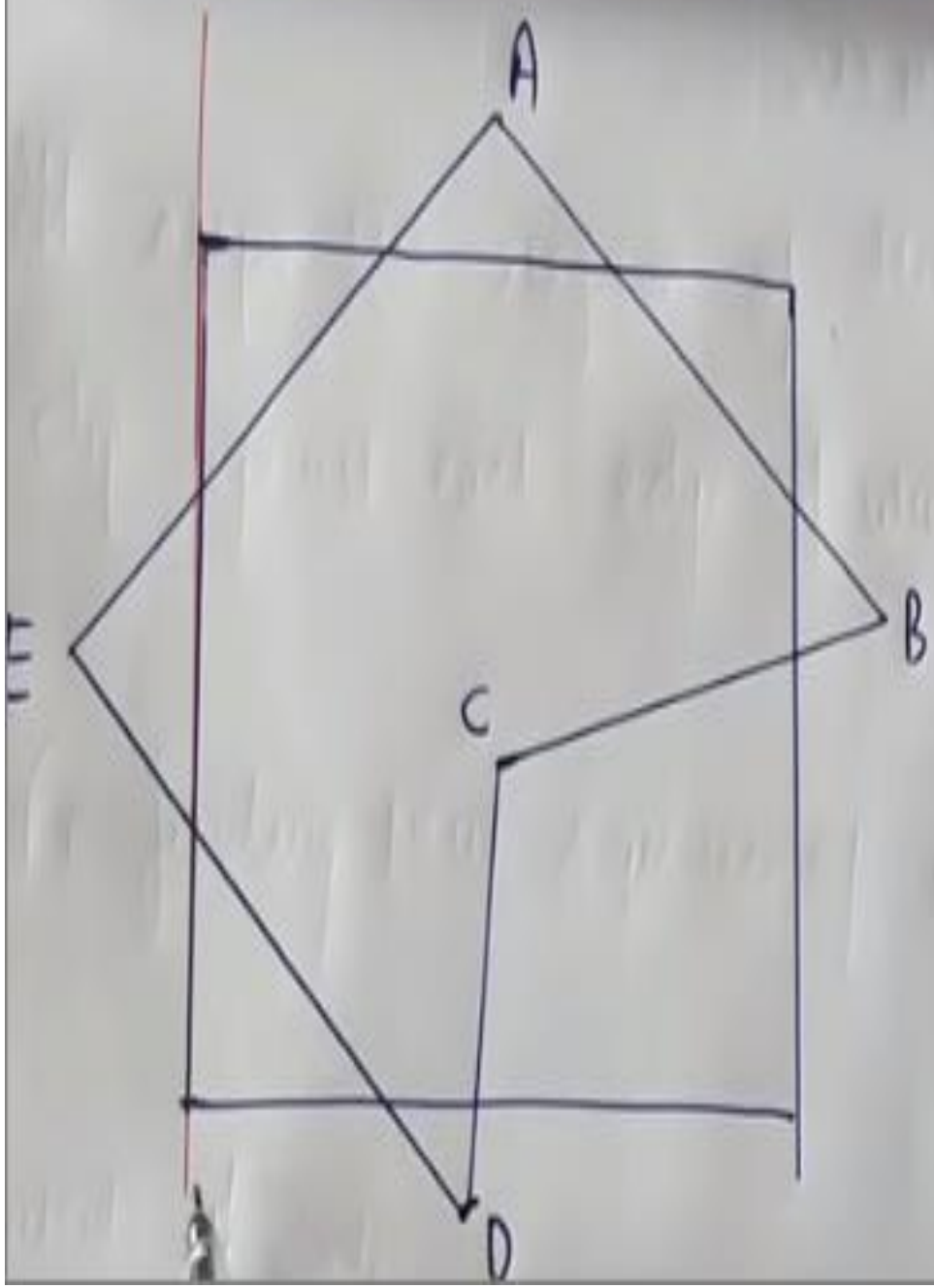
For Top

Edge	Action	Output
B'A	IN-OUT	B''
AA'	OUT-IN	A''A'
A'C''	IN-IN	C''
C''C'	IN-IN	C'
C'B'	IN-IN	B'



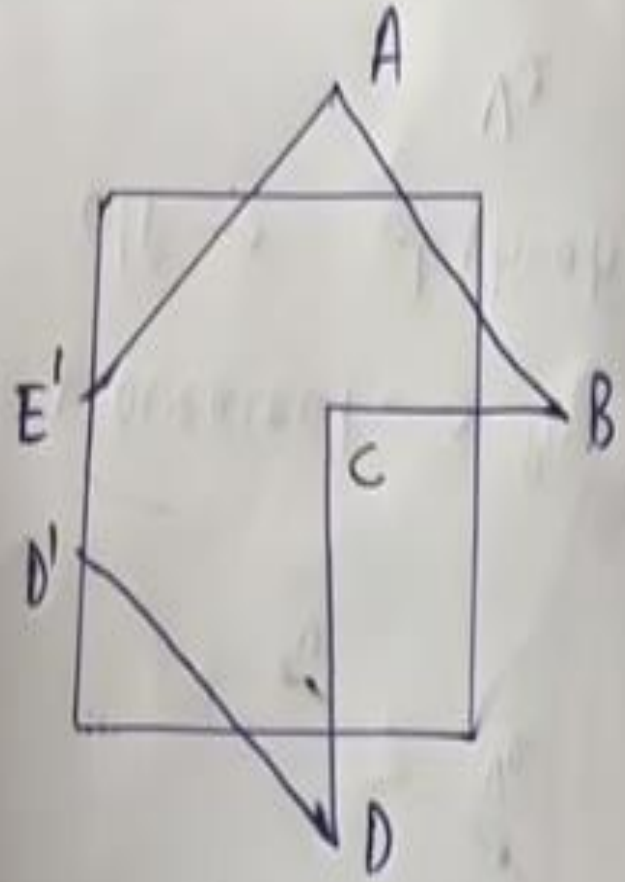

B'
C'
A'
C''

Example 2



① Cliping against left edge

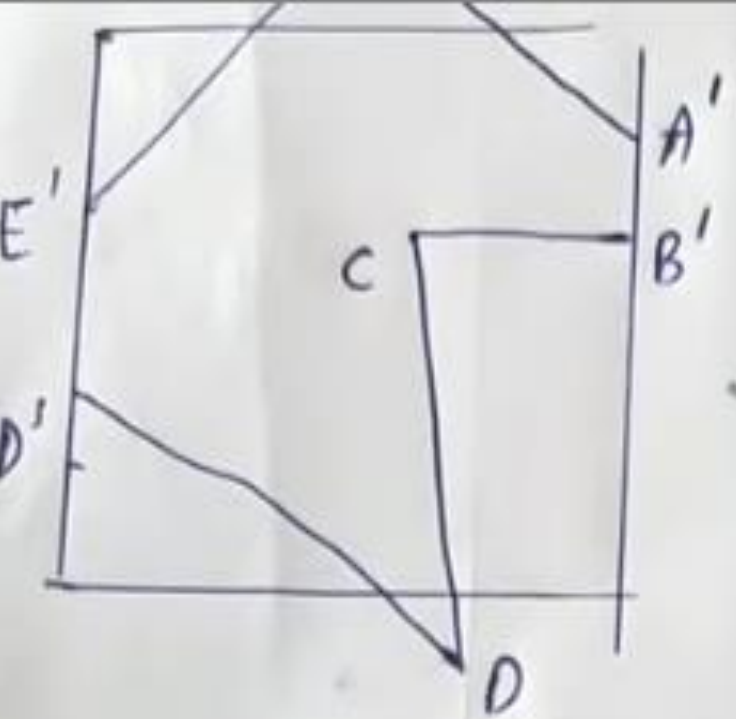
vertex	case	output
AB	in→in	B
BC	in→in	C
CD	in→in	D
DE	in→out	D'
EA	out→in	E'A



clip left

ight edge of window

vertex	case	o/p	
AB	in→out	A'	new vertex
BC	out→in	B'C	new vertex
CD	in→in	D	
DD'	in→in	D'	
D'E'	in→in	E'	
E'A	in→in	A	



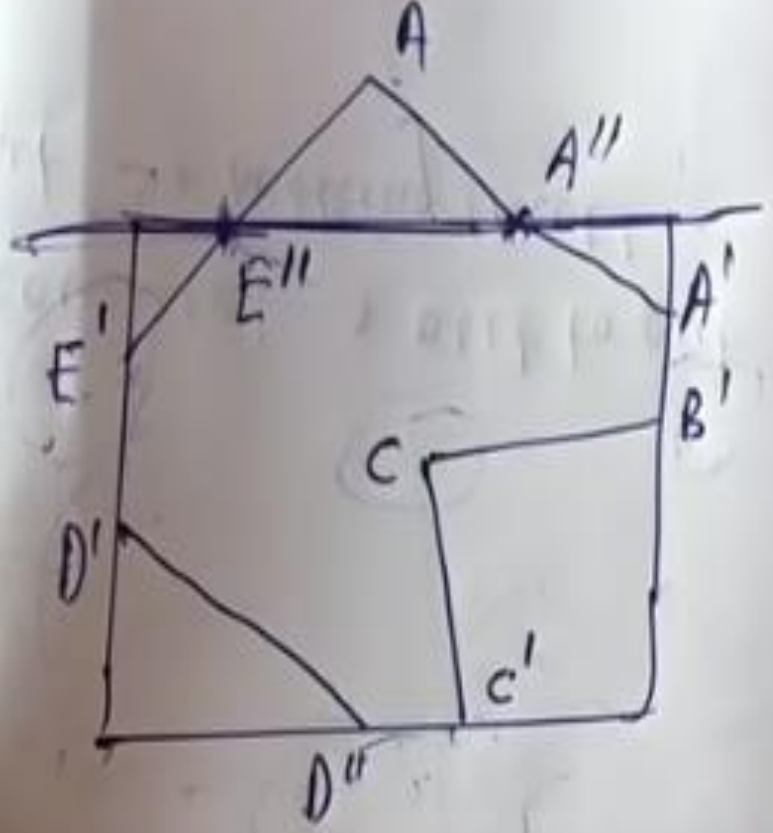
fter Right clip

(3) Bottom clipping

vertex	case	o/p
A A'	in→in	A'
A'B'	in→in	B'
B'C	in→in	C
C D	in→out	C'
D D'	out→in	D'' D'
D'E'	in→in	E'
E'A	in→in	A

② clipping agai

③ Bottom clipping



after bottom clip

④ Top clipping

vertext	case	O/P	
AA'	out → in	A'' A'	→ new vertex
A'B'	in → in	B'	
B'C	"	C	
CC'	"	C'	
C'D''	"	D''	
D''D'	"	D'	
D'E'	"	E'	
E'A	in → out	E''	→ new vertex